AirAuth

Two-Step Air Authentication

Static Hand Gesture + Dynamic Air Signature

Complete Project Documentation

Research Diary content · Setup · Architecture · AI Models

[Team / Group No.] · [Academic Year]

KL University · Capstone Project

Contents

Part	Document
Part A	Research Diary content — Weeks 7, 8, 9 + Technical Evidence pages 47, 49, 51
Part B.1	Project Overview
Part B.2	Setup Guide
Part B.3	01 · System Architecture
Part B.4	02 · Static Gesture Model
Part B.5	03 · Dynamic Signature Model
Part B.6	04 · ML Pipelines
Part B.7	05 · Project Flow
Part B.8	06 · AI Models Explained

RESEARCH DIARY CONTENT

Weeks 7, 8 and 9 · Capstone Project 1 (Semester VII)

AirAuth — Two-Step Air Authentication
Static Hand Gesture + Dynamic Air Signature

Week	Diary phase	Diary pages
Week 7	Phase 3 · Requirements, Concept Selection & Design	Log p. 43 · sections 3.1–3.4 (pp. 30–33)
Week 8	Phase 3 · Requirements, Concept Selection & Design	Log p. 44 · sections 3.5–3.8 (pp. 34–40)
Week 9	Phase 4 · Implementation & Unit Testing (O1–O2)	Log p. 56 · sections 4.1–4.5 (pp. 45–54)
Evidence	Phase 4 · Technical Evidence (O1–O2)	Requirements p. 47 · Data & Model p. 49 · Security & Quality p. 51

How to use: copy each block into the matching box of the diary by hand. Replace every **[...]** with your real names, student IDs, dates and commit IDs. Change the member tasks and percentages if your team's real work differed. Never fill the **Guide Feedback** box yourselves — the guide writes it during the meeting.

The five objectives (diary pp. 9 & 14) — referenced every week

ID	Objective	Measurable success criterion	Sem
O1	Static hand-gesture capture & check (Step 1)	21 hand points captured in the browser; gesture error rate (EER) ≤ 2% against other people	VII
O2	Air-signature capture & check (Step 2)	Fingertip path captured; signature error rate (EER) ≤ 15% on recorded data (8 registration samples)	VII
O3	AI models + training / calibration pipelines	Two trained neural networks; one-command training; calibrated thresholds saved	VIII
O4	MERN website with two-step login	Register → Step 1 → Step 2 → access granted; lockout, rate limit, login tokens; end-to-end tests pass	VIII
O5	Evaluation, security & real-user testing	FAR / FRR / EER / ROC-AUC reports; replay & step-skip protection; ≥ 10 real users tested	VIII

WEEK 07 — Phase 3 — Requirements, Concept Selection & Design

Weekly Progress Log (diary p. 43)

Week No.	07
Phase	Phase 3 — Requirements, Concept Selection & Design
Dates (from – to)	[dd/mm/yyyy – dd/mm/yyyy]
Planned agenda / focus for the week	Act on the Review 1 feedback. Write the requirements specification (3.1), list the constraints and standards (3.2), come up with candidate solutions (3.3), and score them in a decision matrix (3.4).

Member-wise work done this week

Member (Name)	Student ID	Tasks undertaken this week	Prog. %
1. [Name]	[ID]	Wrote the AI requirements: accuracy targets, response time, number of registration samples. Studied what MediaPipe hand tracking outputs (21 hand points, left/right hand).	100
2. [Name]	[ID]	Wrote the security and data requirements: steps in fixed order, account lockout, login tokens, deleting user data. Listed the standards that apply (ISO 29148, ISO 25010, ISO 27001/OWASP, DPDP Act).	100
3. [Name]	[ID]	Wrote the usability requirements: countdowns, auto-stop, retry messages. Drafted 3 candidate designs and built the scoring matrix with the team.	100

Actionable items completed

- Requirements specification written: 16 requirements, each linked to O1–O5.
- Constraints and standards table completed.
- Five candidate designs listed, and the top three scored in the decision matrix — Concept B selected.

Challenges faced & how they were addressed

- **Problem:** there is no public dataset of people's personal hand gestures to train on.
  Solution: train the gesture AI on computer-generated hand poses, and keep the real recordings for testing only.
- **Problem:** we have real signature recordings from only 3 users.
  Solution: added a requirement for the system to learn from each successful login, and for a tool that exports newly collected data so the models can be retrained.

Evidence produced this week

- Requirements table (diary p. 30).
- Decision matrix (diary p. 33).
- Draft system architecture document: Architecture/01_System_Architecture.md
- Commit ID: [commit id]

Guide feedback / comments

Leave this box blank — your guide writes it. Points to raise in the meeting: • Approval of the five objectives and their targets. • Is training on computer-generated data acceptable if testing uses real recordings? • Confirm the list of standards.

Action plan for next week

Task	Expected outcome	Owner	Due date
Draw the architecture, data-flow and component diagrams	Sections 3.5(a), (b), (c) done	[Member 2]	[date]
Detailed design of O1 and O2	Section 3.6 table done	[Member 1]	[date]
Risk register and work plan	Sections 3.7 and 3.8 done	[Member 3]	[date]

3.1 Requirements specification (diary p. 30)

Req. ID	Type	Requirement	Traces to
FR-01	Functional	Capture the 21 hand points in every frame from a normal webcam, inside the browser	O1
FR-02	Functional	Register a hand gesture from 5 captures of the same pose	O1

Req. ID	Type	Requirement	Traces to
FR-03	Functional	Verify the hand gesture; reject the wrong hand and a different gesture	O1
FR-04	Functional	Record the index fingertip's path (time, x, y), stopping automatically when the finger rests	O2
FR-05	Functional	Register an air signature from 8 samples	O2
FR-06	Functional	Verify the air signature using both its shape and how it is drawn	O2
FR-07	Functional	Login must go Step 1 then Step 2; Step 2 cannot be skipped	O4
FR-08	Functional	Send unregistered usernames to the registration page	O4
FR-09	Functional	Users can view their login history and delete all their biometric data	O4
FR-10	Functional	Retrain and recalibrate the AI models from collected data with one command	O3
NFR-01	Non-functional (performance)	Each verification responds in under 1 second	O1, O2
NFR-02	Non-functional (accuracy)	Gesture error rate (EER) ≤ 2%; signature error rate (EER) ≤ 15%	O1, O2, O5
NFR-03	Non-functional (security)	Lock the account after 5 failures; limit request rates; reject replayed data	O4, O5
NFR-04	Non-functional (privacy)	Video never leaves the browser; stored biometric profiles are never sent to the browser	O4
NFR-05	Non-functional (hardware)	Works with a normal laptop webcam on a regular CPU (no graphics card)	O1, O2
NFR-06	Non-functional (maintainability)	Separate services; automated tests for the AI and the API	O3, O4

3.2 Constraints & standards (diary p. 31)

Constraint / standard	Type	How it shapes the design
Normal webcam with no depth sensor	Hardware	Use MediaPipe's flat (2-D) hand points; treat depth as unreliable
CPU-only machines	Hardware	Keep AI models small (about 100,000 parameters) and optimised for speed
Only 3 real users of signature data	Data	Train on computer-generated data; keep the real data for testing
Two-semester timeline	Schedule	Build the simpler checks first (O1, O2 in CP1), add the neural networks in CP2 (O3)
ISO/IEC/IEEE 29148	Standard	Requirements written with IDs and linked to objectives
ISO/IEC/IEEE 42010	Standard	Architecture described in three views
ISO/IEC 25010	Standard	Quality goals: performance, security, usability, maintainability
ISO/IEC 27001 / OWASP Top 10	Standard	Login tokens, rate limiting, input checking, secrets kept out of the code
ISO/IEC/IEEE 29119	Standard	Test plan from unit to integration to full system, with test IDs
India DPDP Act 2023 / GDPR	Legal	User consent, data used only for login, right to delete biometric data
ACM/IEEE-CS Code of Ethics	Ethics	Report accuracy limits honestly; no real banking data

3.3 Candidate solutions (diary p. 32)

Concept	Description	Pros	Cons
A	Air signature only, as a desktop app (our earlier research prototype)	Simple; already built	Only one factor; not a website; no user accounts
B (selected)	Two steps (hand gesture, then air signature). React website with hand tracking in the browser; Express + MongoDB backend; separate Python AI service	Two factors; video stays on the user's device; any webcam works; built in separate parts	More parts to build and connect
C	Upload video to the server and run a large deep-learning model on the raw frames	Could be very accurate with lots of data	Needs a big dataset and a graphics card; privacy risk; slow

Concept	Description	Pros	Cons
D	Password + air signature	Familiar to users	Keeps passwords, which is the problem we want to remove
E	Gesture login with a depth camera or Leap Motion sensor	Accurate 3-D hand data	Needs special hardware most users don't have

3.4 Decision / trade-off matrix (diary p. 33)

Scores run from 1 (poor) to 5 (excellent). Weighted total = sum of (weight × score).

Criterion	Weight	Concept A	Concept B	Concept C
Security (factors, attack resistance)	0.20	2	5	3
Accuracy with little data	0.15	3	4	1
Privacy	0.15	4	5	1
Hardware cost / accessibility	0.10	4	5	2
Speed	0.10	4	4	2
Feasible in 2 semesters	0.15	5	4	2
Usability / demo value	0.10	2	5	3
Easy to scale and maintain	0.05	2	4	3
Weighted total	**1.00**	**3.35**	**4.60**	**1.95**

Selected concept & justification: Concept B scored highest (4.60). It uses two independent factors: a secret hand pose (something you know) and the way you sign in the air (something you are). Video never leaves the user's device, and it works with any ordinary webcam. It can also be built step by step: the simpler gesture and signature checks (O1, O2) in CP1, and the neural network models (O3) in CP2.

WEEK 08 — Phase 3 — Requirements, Concept Selection & Design

Weekly Progress Log (diary p. 44)

Week No.	08
Phase	Phase 3 — Requirements, Concept Selection & Design
Dates (from – to)	[dd/mm/yyyy – dd/mm/yyyy]
Planned agenda / focus for the week	Design the whole system as three diagrams. Write the detailed design of O1 and O2, the risk register and the work plan. Set up the code repository.

Member-wise work done this week

Member (Name)	Student ID	Tasks undertaken this week	Prog. %
1. [Name]	[ID]	Detailed design of O1: cleaning the hand data, the 115 hand measurements, and the matching method. Detailed design of O2: cleaning the fingertip path, signature matching (DTW) and the habit statistics.	100
2. [Name]	[ID]	Drew the high-level architecture and component diagrams. Designed the MongoDB collections. Set up the repository and folder structure (client/, server/, ml-service/).	100
3. [Name]	[ID]	Drew the data-flow diagram (camera → hand points → server → AI → decision). Wrote the risk register and the work plan for weeks 9–15 and CP2.	100

Actionable items completed

- Three architecture diagrams drawn: high-level, data-flow, component interaction (3.5).
- Detailed design table for O1 and O2 completed (3.6).
- Risk register with 8 risks (3.7) and work plan (3.8) completed.
- Git repository set up with the agreed folder structure.

Challenges faced & how they were addressed

- **Problem:** should the AI run in Node.js or Python?
  Solution: in a separate Python service. The Express backend is the only part users talk to, and it calls the AI service using a secret key.
- **Problem:** hands are different sizes and at different distances from the camera.
  Solution: every hand is converted into its own coordinate system based on the palm, so size, position and tilt no longer matter.

Evidence produced this week

- Architecture diagrams (diary pp. 35–37).
- System architecture document: Architecture/01_System_Architecture.md
- Repository set-up commit: [commit id]

Guide feedback / comments

Leave this box blank — your guide writes it. Points to raise in the meeting: • Ask the guide to review the three diagrams before coding starts. • Agree how many unit tests are expected for the Preliminary Design Review.

Action plan for next week

Task	Expected outcome	Owner	Due date
Write the hand-data cleaning and the 115 hand measurements	Code files preprocessing.py and features.py	[Member 1]	[date]
Write the gesture matcher (register + verify)	Code file verifier.py	[Member 1]	[date]
Hand tracking working in the browser	Live hand skeleton on screen	[Member 3]	[date]
Write the unit test plan	Test IDs UT-01 to UT-15	[Member 2]	[date]

3.5 System architecture (diary p. 34) — short note

The system has three parts. The **website** (React) runs hand tracking in the browser. The **backend** (Express + MongoDB) manages users and the two-step login. The **AI service** (Python) checks the gesture and the signature. Only hand coordinates leave the user's computer, never video.

3.5(a) High-level architecture diagram (diary p. 35) — draw this

```
       [ User + Webcam ]
              |  video (stays in the browser)
              v
+------------ WEBSITE (React, in browser) ------------+
| MediaPipe hand tracking -> 21 hand points per frame |
| Gesture capture screen  .  Air-signature screen     |
+----------------------+-----------------------------+
                       | hand coordinates only (JSON)
                       v
+-------- BACKEND (Express, Node.js) --------+     +-----------+
| Registration . Two-step login              |<--->|  MongoDB  |
| Account lockout . Login tokens . Logs      |     | database  |
+----------------------+---------------------+     +-----------+
                       | secret service key
                       v
+-------- AI SERVICE (Python) --------+
| Step 1: gesture checker             |
| Step 2: air-signature checker       |
| Trained models + thresholds         |
+-------------------------------------+
```

Legend: boxes = separate programs; arrows = data moving between them. Video never leaves the browser.

3.5(b) Data-flow / block diagram (diary p. 36)

```
Camera frame -> MediaPipe -> 21 hand points
   |
   +-- STEP 1 (gesture)
   |     15 frames -> clean + "is the hand still?" check
   |     -> convert to palm coordinates -> 115 hand measurements
   |     -> compare with registered gesture -> score
   |     -> score below threshold?  ->  MATCH / NO MATCH
   |
   +-- STEP 2 (air signature)
         fingertip position + time, every frame
         -> remove still start/end -> remove tracking glitches
         -> resample to 128 points in time -> normalise size and position
         -> DTW shape comparison + 12 habit statistics -> combined score
         -> score below threshold?  ->  ACCESS GRANTED / DENIED
```

3.5(c) Component-interaction diagram (diary p. 37)

Draw four columns — **Website | Backend | AI service | Database** — and these numbered arrows:

#	From → To	Message
1	Website → Backend	"Start login" with the username
2	Backend → Database	Create a login session at Step 1
3	Website → Backend	Gesture sample
4	Backend → AI service	Verify the gesture
5	AI service → Backend	Result (match / no match + score)
6	Backend → Database	Save the attempt; move the session to Step 2
7	Website → Backend	Air-signature sample
8	Backend → AI service	Verify the signature
9	AI service → Backend	Result
10	Backend → Database	Save the attempt, update the profile, close the session
11	Backend → Website	Login token — ACCESS GRANTED

3.6 Detailed design of CP1 objectives O1–O2 (diary p. 38)

Component	Description	Inputs	Outputs	Dependencies
Hand tracker (website)	MediaPipe finds one hand in each video frame	Webcam frames	21 points (x, y, z) + left/right hand	MediaPipe
Gesture cleaning (O1)	Removes bad frames, corrects the camera's width/height ratio, rejects a moving hand, mirrors left hands	15 frames	Clean hand frames	NumPy

Component	Description	Inputs	Outputs	Dependencies
Gesture measurements (O1)	Converts the hand into palm coordinates; measures joint angles, fingertip distances, finger spread, palm direction, bone proportions	Clean frames	115 numbers (middle value over the frames)	Gesture cleaning
Gesture checker (O1)	Checks the 5 registration samples match each other; compares a new gesture with them; rejects wrong hand and replays	Saved profile + new gesture	Accept/reject + score	Gesture measurements
Path cleaning (O2)	Removes still start/end, fixes tracking glitches, resamples to 128 points in time, normalises size and position	Fingertip points with time	128-point path	NumPy
Signature measurements (O2)	9 values per point (position, speed, direction, curvature…) and 12 habit statistics (duration, pauses…)	128-point path	Measurement tables	Path cleaning
Shape matcher, DTW (O2)	Compares paths while allowing for drawing faster or slower in places; uses the closest registered signature	New path + registered paths	Distance	Path cleaning
Signature checker (O2)	Adjusts for each user's own variation, combines scores, applies threshold, rejects replays	Saved profile + new signature	Accept/reject + score	DTW, measurements

3.7 Risk register (diary p. 39)

Risk ID	Description	Likelihood	Impact	Mitigation	Owner
R1	Too little real data to train the neural network models	High	High	Train on computer-generated data; use real data for testing	[Member 1]
R2	Hand tracking jitters or jumps	High	Medium	Automatic glitch correction; middle value over many frames	[Member 1]
R3	Bad lighting or hand outside the camera view	Medium	Medium	"Hand detected" indicator; clear retry messages	[Member 3]
R4	Someone watches and copies the gesture	Medium	High	Step 2 checks how you sign, which is hard to copy	[Member 1]
R5	Attacker re-sends recorded hand data	Low	High	Reject samples identical to stored ones; limit request rates	[Member 2]
R6	Verification too slow on a normal CPU	Medium	Medium	Small models; fast vectorised code; target under 1 second	[Member 1]
R7	Leak of biometric data	Low	High	No video upload; profiles stay on the server; "delete my data" option	[Member 2]
R8	Delays connecting the three parts	Medium	Medium	Agree the data format early; end-to-end test script	[Member 2]

3.8 Project plan / work breakdown (diary p. 40)

Phase / Task	Owner(s)	Start	End	Dependencies
O1: hand-data cleaning + 115 measurements	[Member 1]	Week 9	Week 9	Design (3.6)
O1: gesture checker + unit tests	[Member 1], [Member 2]	Week 9	Week 10	Measurements
Hand tracking in the browser	[Member 3]	Week 9	Week 10	—
O2: path cleaning + DTW	[Member 1]	Week 10	Week 11	Recorded signatures
O2: signature checker + testing on recorded data	[Member 1], [Member 2]	Week 11	Week 12	DTW
Connect O1 + O2 (section 5.1)	All	Week 13	Week 14	O1, O2
Standards checklist + Preliminary Design Review preparation	All	Week 14	Week 15	—
CP2: neural network models (O3), full website (O4), evaluation (O5)	All	CP2 Wk 1	CP2 Wk 13	CP1 work

WEEK 09 — Phase 4 — Implementation & Unit Testing (O1–O2)

Weekly Progress Log (diary p. 56)

Week No.	09
Phase	Phase 4 — Implementation & Unit Testing (O1–O2)
Dates (from – to)	[dd/mm/yyyy – dd/mm/yyyy]
Planned agenda / focus for the week	Start building O1, the hand gesture check: the hand-data cleaning, the 115 hand measurements and the gesture checker. Get hand tracking working live in the browser. Write the unit test plan.

Member-wise work done this week

Member (Name)	Student ID	Tasks undertaken this week	Prog. %
1. [Name]	[ID]	Wrote the hand-data cleaning code (preprocessing.py): reads the frames, corrects the camera's width/height ratio, rejects a moving hand. Wrote the 115 hand measurements (features.py). Started the gesture checker (verifier.py).	70
2. [Name]	[ID]	Wrote the unit test plan (15 tests). Set up the testing tool (pytest). Wrote and ran the first 4 tests.	60
3. [Name]	[ID]	Got hand tracking working in the browser: webcam + MediaPipe drawing the 21-point hand skeleton live (CameraStage.jsx, handLandmarker.js).	60

Actionable items completed

- The 115 hand measurements work correctly: a straight finger measures about 0–15° at its joints, a curled finger about 80–100°.
- The same gesture repeated scores about 0.6 apart; a different gesture about 3.0 apart — clearly separable.
- The live hand skeleton shows in the browser.
- 4 unit tests written and passing (UT-01, UT-02, UT-08, UT-09).

Challenges faced & how they were addressed

- **Problem:** our test data kept failing the "is the hand still?" check.
  Solution: the test-data generator was moving the hand to a new spot in every frame. We fixed the generator to keep the hand in one place during a capture. The check itself was correct.
- **Problem:** the same gesture gave different numbers when the hand was tilted.
  Solution: every hand is now turned into its own palm-based coordinates before measuring, which removed the effect of tilt.

Evidence produced this week

- Code files: ml-service/airauth/static_gesture/preprocessing.py, features.py, verifier.py (started)
- Browser code: client/src/components/CameraStage.jsx, client/src/lib/handLandmarker.js
- Tests UT-01, UT-02, UT-08, UT-09 (ml-service/tests/test_static_gesture.py)
- Commit IDs: [commit ids]

Guide feedback / comments

Leave this box blank — your guide writes it. Points to raise in the meeting: • Show the live hand skeleton and the measurement values. • Confirm the O1 accuracy target (error rate ≤ 2%) is acceptable.

Action plan for next week

Task	Expected outcome	Owner	Due date
Finish the gesture checker (register, verify, wrong hand, replay)	All O1 tests pass	[Member 1]	[date]
Start the signature path cleaning (O2)	Code file dynamic_signature/preprocessing.py	[Member 1]	[date]
Gesture capture screen with countdown	Code file StaticGestureCapture.jsx	[Member 3]	[date]
Run and record the remaining O1 tests	Results in table 4.4	[Member 2]	[date]

4.1 Implementation progress (diary p. 45)

Task	Owner(s)	Start	End	% done	Dependencies
Hand-data cleaning	[Member 1]	Week 9	Week 9	100	—
115 hand measurements	[Member 1]	Week 9	Week 9	100	Hand-data cleaning
Gesture checker	[Member 1]	Week 9	Week 10	50	Measurements
Hand tracking in the browser	[Member 3]	Week 9	Week 10	60	MediaPipe
O1 unit tests	[Member 2]	Week 9	Week 10	40	Gesture checker
Signature path cleaning (O2)	[Member 1]	Week 10	Week 11	0	—

Update the % column each week as work progresses.

4.2 Deviations from design (diary p. 46)

If nothing changed from the Week 8 design, write **"No deviations this week."** If you want an entry, this one is accurate:

Objective / module	Deviation from design	Reason	Impact
O1 / test data	Added a computer-generated hand model to create test gestures	No public dataset of personal gestures exists	Lets us test O1 without real users; real users are still needed later (O5)

Build & Version Control (diary p. 50) — start this table now

Module / component	Owner	Repository / commit ref	Build status
Gesture cleaning + measurements (O1)	[Member 1]	[commit id]	Tests passing
Browser hand tracking	[Member 3]	[commit id]	Builds and runs in browser
Unit test set-up (pytest)	[Member 2]	[commit id]	4 / 4 passing

4.3 Unit test plan (diary p. 52)

The test IDs line up with the real tests already in ml-service/tests/.

Test-case ID	Description	Objective / module	Test type
UT-01	Measurements have the right size (115 numbers; 89 for the AI)	O1 / measurements	Unit
UT-02	Moving or resizing the hand doesn't change the measurements	O1 / measurements	Unit
UT-03	The correct gesture is accepted (at least 4 of 5 tries)	O1 / gesture checker	Unit
UT-04	A different gesture is rejected	O1 / gesture checker	Unit
UT-05	The wrong hand is rejected	O1 / gesture checker	Unit
UT-06	An exact copy of a stored sample is rejected	O1 / gesture checker	Security
UT-07	Registration finds the one sample that is a different gesture	O1 / gesture checker	Unit
UT-08	A moving hand is rejected	O1 / hand-data cleaning	Unit
UT-09	Too few frames are rejected	O1 / hand-data cleaning	Unit
UT-10	Signature path becomes 128 points and is normalised	O2 / path cleaning	Unit
UT-11	A tracking glitch is found and fixed	O2 / path cleaning	Unit
UT-12	DTW gives 0 for identical paths and the same result both ways	O2 / DTW	Unit
UT-13	The owner's signature is accepted; other people's are rejected	O2 / signature checker	Unit
UT-14	A replayed signature is rejected	O2 / signature checker	Security
UT-15	A too-short signature is rejected	O2 / path cleaning	Unit

4.4 Unit test execution (diary p. 53) — Week 9 entries only

Record only the tests you actually ran this week:

Test-case ID	Input	Expected output	Actual output	Pass/Fail
UT-01	One gesture capture (10 frames)	115 numbers; 89 for the AI	115; 89	Pass
UT-02	Same hand frame, enlarged 1.7× and shifted	Same measurements	Same (difference < 0.000001)	Pass

Test-case ID	Input	Expected output	Actual output	Pass/Fail
UT-08	8 frames with the hand in a new place each frame	Error: "hand moved"	Error: "hand moved"	Pass
UT-09	No frames	Error: "too few frames"	Error: "too few frames"	Pass

Add UT-03 to UT-15 in weeks 10–12 as you run them.

4.5 Issue tracker (diary p. 54)

Issue no.	Description	Impact	Owner	Status	Guide sign
I-01	Test data failed the "hand still?" check because the generator moved the hand every frame	Medium (blocked testing)	[Member 1]	Closed: generator fixed	(guide)
I-02	The same gesture gave different numbers when the hand was tilted	High (wrong matches)	[Member 1]	Closed: palm-based coordinates added	(guide)

Technical Evidence — CSE & Allied (CP1 · objectives O1–O2)

These pages are filled during Phase 4 (weeks 9–12) and checked at the Preliminary Design Review. Every number below comes from the project's real test runs (ml-service/reports/ and the test suite). If your guide wants CP1 pages limited to O1–O2 only, keep the rows marked O1/O2 and move the O4 rows to the CP2 pages (pp. 79–83).

Requirements & Architecture — ISO/IEC/IEEE 29148 · 42010 (diary p. 47)

Req. ID	Type (F/NF)	Requirement / quality attribute	Priority	Traces to
FR-01	F	Capture the 21 hand points in every frame from a normal webcam, inside the browser	High	O1
FR-02	F	Register a hand gesture from 5 captures of the same pose; reject a set where one capture is a different gesture	High	O1
FR-03	F	Verify the hand gesture; reject the wrong hand, a different gesture and an exact replay	High	O1
FR-04	F	Record the index fingertip's path (time, x, y) and stop automatically when the finger rests	High	O2
FR-05	F	Register an air signature from 8 samples; reject a badly captured sample	High	O2
FR-06	F	Verify the air signature using shape (DTW) and drawing habits (speed, pauses, duration)	High	O2
FR-07	F	Give a clear retry message when a capture is unusable (too short, hand moved, too few frames)	Medium	O1, O2
NFR-01	NF · Performance	Each verification responds in under 1 second on a normal CPU	High	O1, O2
NFR-02	NF · Accuracy	Gesture error rate (EER) ≤ 2%; signature error rate (EER) ≤ 15%	High	O1, O2
NFR-03	NF · Security	Reject replayed (copied) samples	High	O1, O2
NFR-04	NF · Privacy	Only hand coordinates are processed — never video	High	O1, O2
NFR-05	NF · Portability	Works with an ordinary laptop webcam, no special hardware	Medium	O1, O2
NFR-06	NF · Maintainability	Each module has automated unit tests	Medium	O1, O2
NFR-07	NF · Robustness	Works regardless of hand size, position and tilt; corrects tracking glitches	High	O1, O2

Architecture view for this page: reuse the high-level diagram from section 3.5(a) (diary p. 48 — quality attributes covered: performance, security, usability, maintainability, portability).

Data & Model Evidence (diary p. 49)

Dataset(s)

Dataset / source	Size & split	Licence & provenance	Pre-processing / issues
Research air-signature dataset (recorded by the team with a desktop webcam app, 1280×720)	3 users × 3 sessions × 10 = 90 signatures. Split by session: register on session 1 (8 signatures), test on sessions 2–3.	Own data, collected from consenting volunteers [confirm names/consent forms]. Not shared publicly.	Fingertip path only. Glitch removal, idle trimming, resampling to 128 points. Issue: only 3 users, and they drew simple shapes.
Computer-generated hand poses (our own kinematic hand model)	2,048 pose types: 1,638 for training, 410 held out for testing. 150 simulated users for evaluation.	Generated by our code (synthetic_hand.py) — no licence restrictions.	Random hand sizes, angles (±35°) and tracking noise. Issue: synthetic, so real-world error may be higher.
MediaPipe Hand Landmarker (pre-trained model by Google)	Used as-is; not retrained.	Apache 2.0 licence (MediaPipe).	Gives 21 hand points per frame. Issue: jitter and occasional jumps, handled in cleaning.

Experiments

Run ID	Config / model	Metric	Baseline	Result vs target
EXP-01	Signature DTW using x,y only vs x,y + depth (z)	EER	x,y,z: 30.8%	x,y only: 14.4% — depth removed from the design

Run ID	Config / model	Metric	Baseline	Result vs target
EXP-02	Signature DTW: compare to closest sample vs average of 3 closest vs average of all (5 registration samples)	EER	Average of all: 27–49%	Closest sample: 18–22% — chosen
EXP-03	Number of registration signatures: 5 vs 10 vs 20 (two sessions)	EER	5 samples: 18–22%	10: 8–12%; 20: 3–6% — registration set to 8, plus learning from logins
EXP-04	Gesture template matcher, 150 simulated users, unseen gestures	EER	Target ≤ 2%	1.2% — target met
EXP-05	Signature check: DTW + habit statistics, 8 registration samples, later sessions	EER	DTW alone: 13.3%	8.7% — target ≤ 15% met
EXP-06	Verification speed (one request, 8 / 20 stored signatures)	Latency	Target < 1 s	≈ 140 ms / ≈ 470 ms — target met
EXP-07	Gesture check vs someone who copies the exact gesture	EER	—	37.7% — expected limitation; Step 2 covers it

Bias / fairness & data-privacy check (consent, anonymisation, DPDP/GDPR relevance)

- **Small, narrow sample:** only 3 real users, so results may not hold for everyone (different hand sizes, skin tones, ages, or left-handed users). Plan: test with 10–20 varied volunteers in CP2 (O5).
- **Hand tracking fairness:** MediaPipe's accuracy can vary with lighting and skin tone. We added a "hand detected" indicator and clear retry messages; to be measured across users in CP2.
- **Left-handed users:** left hands are mirrored into the same layout, so both hands are supported.
- **Consent:** volunteers agree before their signatures are recorded [attach consent forms].
- **Data minimisation:** only hand coordinates are stored — no video, no face images.
- **Anonymisation:** exported training data uses random user IDs, not names; raw data is excluded from the Git repository.
- **DPDP Act 2023 / GDPR:** data is used only for login; users can delete their account and all biometric data.

Security & Quality — OWASP / ISO 27001 · ISO/IEC 25010 (diary p. 51)

Control / quality attribute	Applicable? (Y/N)	How addressed / evidence
Replay attack protection (O1, O2)	Y	A sample identical to a stored one is rejected. Tests UT-06 and UT-14.
Input validation — OWASP A03 Injection (O1, O2)	Y	Every request is checked for shape and size before processing (schemas.py); malformed data returns an error, never crashes.
Privacy of biometric data — ISO 27001 / DPDP (O1, O2)	Y	No video leaves the browser; stored profiles never sent back to the browser; "delete my data" option.
Service access control — OWASP A01 (O1, O2)	Y	The AI service only answers requests carrying the secret service key (security.py); test test_biometric_endpoints_require_service_key.
Brute-force protection — OWASP A07 (O4)	Y	3 attempts per step, account locked after 5 failures, request rate limits.
Step order cannot be skipped — OWASP A01 (O4)	Y	Server tracks which step each login is on; Step 2 rejected unless Step 1 passed (end-to-end test).
Secrets & configuration — OWASP A05	Y	Secrets kept in .env (not in Git); security headers (helmet); CORS limited to the website.
Vulnerable components — OWASP A06	Y	Library versions pinned; run npm audit and pip check before PDR [to do].
Performance efficiency — ISO 25010	Y	Verification ≈ 140–470 ms (EXP-06); target under 1 s met.
Reliability — ISO 25010	Y	Unusable captures give a retry message; if the AI model is missing, the simpler checks still work.
Usability — ISO 25010	Y	Countdowns, live hand skeleton, "hand detected" indicator, auto-stop, clear messages.
Maintainability — ISO 25010	Y	Separate modules for cleaning, measurements and checking; 23 automated tests passing.

Before copying: replace every [...] with real names, student IDs, dates and commit IDs, and change any task or percentage that doesn't match what your team actually did.

AirAuth — Two-Step Air Authentication

Password-less login with a webcam, in two AI-verified steps:

1. **Static hand gesture** — hold your secret hand pose. 21 MediaPipe key-points → 115 geometric features → template matcher + neural pose embedding.
2. **Dynamic air signature** — sign in the air with your index finger. The trajectory's shape *and* motion → DTW + behavioural statistics + a Conv1D-BiGRU-attention embedding network.

Access is granted only when both steps pass, in order. Unregistered users are sent to registration, which enrolls both factors.

Stack: MongoDB · Express · React · Node.js + a Python FastAPI ML service.

```
client/        React + Vite UI, on-device MediaPipe hand tracking (no video leaves the browser)
server/        Express + Mongoose API: accounts, 2-step login state machine, lockout, audit
ml-service/    Python AI service: preprocessing, features, models, pipelines, tests
Architecture/  Architecture & model documents (start here)
scripts/       setup.ps1, start-dev.ps1
```

Quick start (Windows)

Full step-by-step instructions and troubleshooting: **SETUP.md**.

Prerequisites: Python 3.11, **Node.js 18+**, **MongoDB** (local, Docker or Atlas).

```
powershell -ExecutionPolicy Bypass -File scripts\setup.ps1      # install everything
# edit server\.env  (MONGODB_URI, JWT_SECRET)
powershell -ExecutionPolicy Bypass -File scripts\start-dev.ps1  # start all three services
# open http://localhost:5173
```

Retrain / recalibrate the models (trained models are already included):

```
cd ml-service
.\.venv\Scripts\python.exe -m pipelines.run_full_pipeline
.\.venv\Scripts\python.exe -m pytest tests -q
```

Current results

Step	Data	Result
1 Gesture	150 simulated users, gestures unseen in training	EER 0.1 % vs random impostors
2 Signature	real research dataset, 8-signature enrollment, later sessions	EER 8.3 %, AUC 0.94; FAR 5 % / FRR 17 % at the live threshold

See `ml-service/reports/` for full tables and `Architecture/05_Project_Flow.md` for setup, the demo script and next steps.

Documentation

Document	Content
01_System_Architecture	MERN + ML architecture, flows, data model, security
02_Static_Gesture_Model	Step 1 features, network, decision rule, results
03_Dynamic_Signature_Model	Step 2 preprocessing, features, network, results
04_ML_Pipelines	Training, evaluation, calibration, retraining loop
05_Project_Flow	What to do, in order — setup, run, demo, roadmap
06_AI_Models_Explained	Plain-language guide: every model and technique, why it was chosen, why it beats the alternatives

AirAuth — Setup Guide (Windows)

Step-by-step instructions to get the project running on a new Windows machine. All commands are run in **PowerShell** from the project folder:

```
cd "C:\Files\Projects\Capstone V2"
```

---

1. Prerequisites

Tool	Version	Install command	Check
Python	3.11	https://www.python.org/downloads/ (tick **"Add python.exe to PATH"**)	`python --version`
Node.js (includes npm)	18 or newer	`winget install OpenJS.NodeJS.LTS`	`node --version`
MongoDB	6 or newer	`winget install MongoDB.Server`	`Get-Service MongoDB` → **Running**
Webcam	any	—	—
Browser	Chrome / Edge (recent)	—	—

> **Important:** after installing Node.js or MongoDB, **close and reopen VS Code / PowerShell**. Open terminals keep their old PATH and will report `npm is not recognized`. (The project scripts also reload PATH automatically, but your own terminal will not.)

MongoDB alternatives (pick one if you don't want a local install):

```
# Docker
docker run -d -p 27017:27017 --name airauth-mongo mongo:7

# or MongoDB Atlas (free cluster): copy its connection string into server\.env (step 3)
```

---

2. Install the project dependencies (one time)

```
powershell -ExecutionPolicy Bypass -File scripts\setup.ps1
```

This script:

1. creates the Python virtual environment `ml-service\.venv` and installs `ml-service\requirements.txt`
2. checks that Node.js is available
3. creates `server\.env` from `server\.env.example` (if missing)
4. runs `npm install` in `server\` and `client\`

It takes a few minutes (TensorFlow is the largest download).

▶ **Manual alternative (if you prefer to run each step yourself)**

---

3. Configure the backend

Open `server\.env` and check these values:

Variable	What to set
`MONGODB_URI`	`mongodb://127.0.0.1:27017/airauth` for a local/Docker MongoDB, or your Atlas connection string
`JWT_SECRET`	any long random string (used to sign login tokens)
`ML_SERVICE_URL`	leave as `http://127.0.0.1:8000`
`ML_SERVICE_API_KEY`	leave as is (must match the ML service; default `dev-ml-service-key`)

Generate a random secret if you need one:

```
-join ((1..48) | ForEach-Object { '{0:x2}' -f (Get-Random -Maximum 256) })
```

---

4. AI models

Trained models and their calibration are **already included** in `ml-service\models\` — nothing to do for a first run.

To retrain and recalibrate both models from scratch (≈ 15 minutes):

```
cd ml-service
.\.venv\Scripts\python.exe -m pipelines.run_full_pipeline
cd ..
```

Quick smoke test of the pipelines (small data, ≈ 3 minutes — **overwrites** the models with low-quality ones, so re-run the full pipeline afterwards):

```
.\.venv\Scripts\python.exe -m pipelines.run_full_pipeline --quick
```

---

5. Run the application

Option A — one command

```
powershell -ExecutionPolicy Bypass -File scripts\start-dev.ps1
```

Three windows open:

Window	Service	URL	Ready when you see
1	AI / ML service (FastAPI)	http://127.0.0.1:8000/docs	`Application startup complete`
2	Backend API (Express)	http://localhost:5000/api/system/health	`[db] connected` and `listening on http://localhost:5000`
3	Website (React + Vite)	http://localhost:5173	`Local: http://localhost:5173/`

Option B — three terminals manually

```
# Terminal 1 — AI service
cd ml-service
.\.venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000

# Terminal 2 — backend
cd server
npm run dev

# Terminal 3 — website
cd client
npm run dev
```

Open **http://localhost:5173** and allow camera access. To stop, close the windows (or press `Ctrl+C` in each terminal).

---

6. Check that everything works

1. The home page **status panel** shows API, MongoDB and ML service with green dots, and both model versions.
2. **Register** a user: * pick a username * Step 1: show the **same** secret hand gesture 5 times (same hand, hold it still until the bar fills) * Step 2: sign in the air with your index finger 8 times — hold the finger still for ~1 s to finish each one
3. **Log in** with that username: gesture → air signature → **ACCESS GRANTED** → dashboard.
4. Try logging in with an unknown username — you are redirected to registration.

Automated tests for the AI service:

```
cd ml-service
.\.venv\Scripts\python.exe -m pytest tests -q      # expect: 23 passed
```

---

7. Troubleshooting

Problem	Cause / fix
`npm` / `node` is not recognized	Node.js missing, or the terminal was opened before installing it → install Node.js, then **restart VS Code / PowerShell**
`running scripts is disabled on this system`	Run scripts with `powershell -ExecutionPolicy Bypass -File scripts\...` as shown above
API window: `could not connect to mongodb://...`	MongoDB is not running → `net start MongoDB` (admin terminal), start the Docker container, or fix `MONGODB_URI`
Status panel: ML service *unavailable*	Window 1 is not running or crashed — read its error; make sure port 8000 is free
Status panel: model *not trained*	`ml-service\models\*.keras` missing → run the full pipeline (section 4)
`Address already in use` / port busy	Another copy is still running → close old windows, or find it with `netstat -ano | findstr :5000`
Camera does not start	Allow camera permission in the browser (lock icon in the address bar); close other apps using the webcam

Problem	Cause / fix
"Your hand moved while capturing"	Hold the gesture completely still until the progress bar is full
"The signature was too short"	Draw larger and longer; keep the hand in view until you finish
Gesture/signature keeps failing	Use good lighting, the same hand and the same distance as at registration; re-register if needed
`Account temporarily locked`	5 failed attempts in a row → wait 15 minutes (set in `server\.env`: `LOCKOUT_MINUTES`)
First AI-service start is slow	Normal: TensorFlow loads and compiles both models (≈ 5–15 s)

---

8. Project layout (for orientation)

```
Capstone V2/
├── SETUP.md            ← this guide
├── README.md           ← project overview
├── Architecture/       ← system & model architecture documents
├── client/             ← React website           (npm run dev → :5173)
├── server/             ← Express + MongoDB API     (npm run dev → :5000)
├── ml-service/         ← Python AI service         (uvicorn      → :8000)
│   └── data/research_signatures/  ← recorded research dataset (3 users)
└── scripts/            ← setup.ps1, start-dev.ps1
```

Next: read `Architecture/05_Project_Flow.md` for the demo script and roadmap.

AirAuth — System Architecture

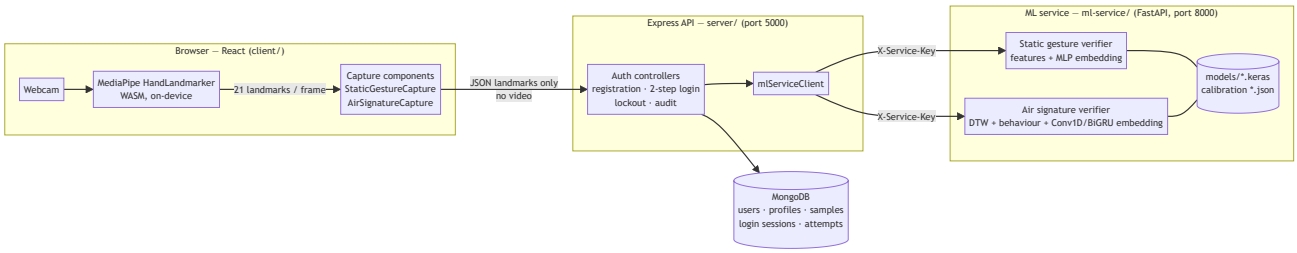
Two-step, password-less authentication with a webcam:

Step	Factor	What the user does	What is verified
1	**Static hand gesture** (knowledge + weak biometric)	Holds a secret hand pose for ~0.5 s	Pose shape (21 key-points → angles, distances, canonical coordinates) + hand geometry
2	**Dynamic air signature** (behavioural biometric)	Draws their signature in the air with the index finger	Trajectory shape, speed, rhythm, timing

Access is granted **only if both steps pass, in order**.

---

1. Deployment view (MERN + ML micro-service)

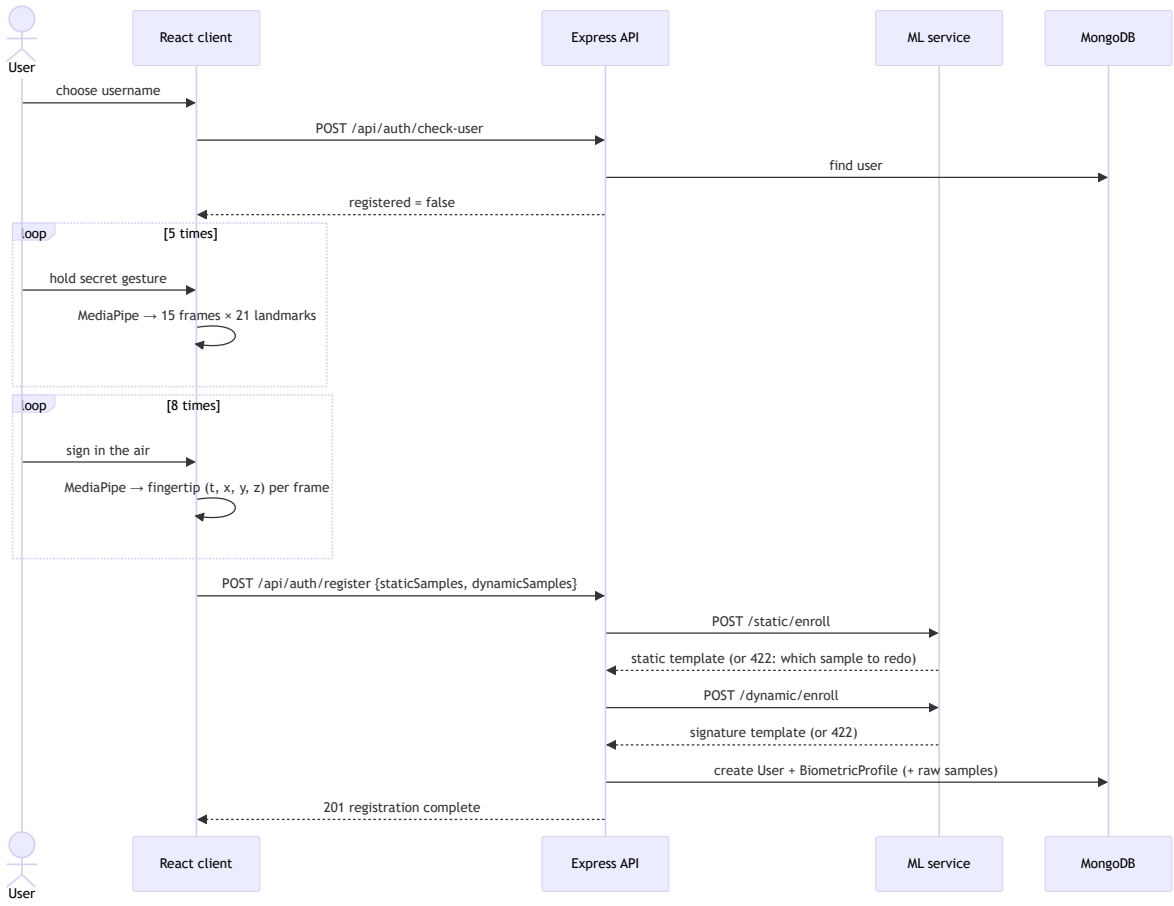


Tier	Folder	Tech	Responsibility
MongoDB	—	MongoDB 6+/Atlas	Users, biometric templates, raw samples (research), login sessions (TTL), audit log
Express	`server/`	Node 18+, Express 4, Mongoose 8, JWT	Accounts, two-step state machine, attempt limits, lockout, tokens, rate limiting
React	`client/`	React 18, Vite, MediaPipe Tasks Vision	Camera, on-device hand tracking, capture UX, wizards, dashboard
Node	`server/`	—	(runtime of the API)
ML	`ml-service/`	Python 3.11, FastAPI, NumPy, TensorFlow/Keras 3	Preprocessing, feature extraction, AI models, scoring, calibration, training pipelines

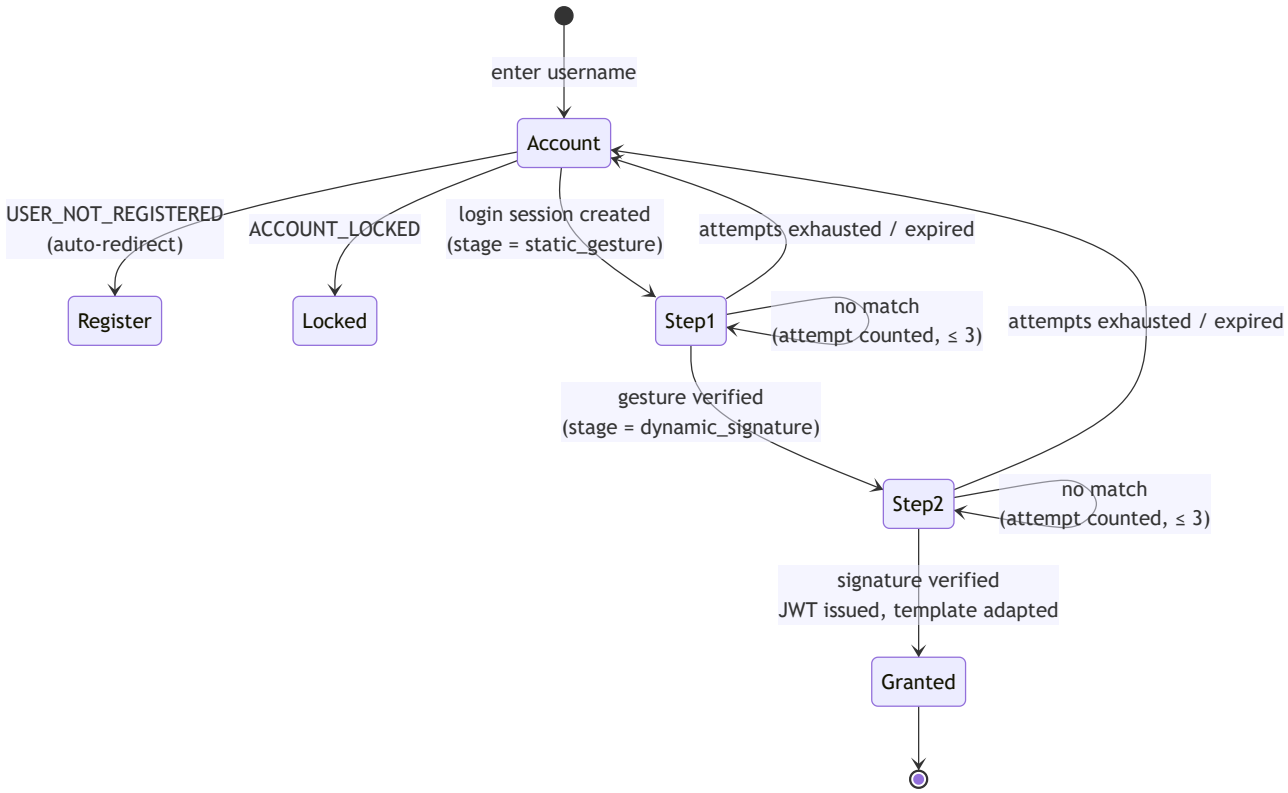
Why a separate Python service? The models, DTW and feature engineering live in the Python ecosystem (and build on the original research preprocessing). Express stays the only public API; the ML service is reachable only with the shared `X-Service-Key`.

---

2. Registration flow



3. Login flow (two-step state machine)





Syntax error in text

mermaid version 10.9.1

4. Data model (MongoDB)

Collection	Key fields	Notes
`users`	username (unique), displayName, failedAttempts, lockUntil, lastLoginAt	No password
`biometricprofiles`	user, staticTemplate, dynamicTemplate, staticHandedness, dynamicAdaptations	Templates are opaque JSON produced by the ML service; never sent to the browser
`biometricsamples`	user, kind (static/dynamic), purpose (enrollment/login), accepted, sample	Raw captures for retraining (`STORE_RAW_SAMPLES`)
`loginsessions`	tokenHash, user, stage, attempts, expiresAt (TTL)	Server-side 2-step state
`authattempts`	user, step, accepted, reason, score, threshold, matchPercent, ip	Audit log / dashboard history

5. Security controls

Threat	Control
Skipping Step 1	Server-side `LoginSession.stage`; Step 2 endpoint rejects sessions not at `dynamic_signature`
Brute force	≤ 3 attempts per step per session, account lock after 5 consecutive failures (15 min), IP rate limits
Replaying stored data	ML rejects probes that are (near-)identical to an enrollment sample (`replay_suspected`)
Shoulder-surfing the gesture	Step 2 is a behavioural biometric — copying the shape does not copy speed/rhythm
Biometric data leakage	Video never leaves the browser; templates never leave the server; `DELETE /api/users/me` removes everything
Direct ML access	ML endpoints require `X-Service-Key`; only Express knows it
Token theft	Short-lived JWT (1 h), login-session tokens stored only as SHA-256 hashes

Known limitation (documented for the capstone): landmarks are computed in the browser, so a malicious client could submit synthetic landmark data. Production hardening would add server-side liveness checks (e.g. random challenge gestures) or on-server landmark extraction from video.

6. Repository layout

```
Capstone V2/
├── Architecture/                ← these documents
├── client/                      ← React + Vite front-end
│   └── src/{api,components,config,context,lib,pages,styles}
├── server/                      ← Express + MongoDB API
│   ├── src/{config,controllers,middleware,models,routes,services,utils}
│   └── scripts/exportBiometricSamples.js
├── ml-service/                  ← Python AI service
│   ├── airauth/                 ← core library
│   │   ├── common/              (config-free helpers: fusion, metrics, model registry, hand
topology)
│   │   ├── static_gesture/      (preprocessing, features, synthetic hands, MLP, verifier)
│   │   ├── dynamic_signature/   (preprocessing, features, DTW, synthetic signatures, Conv1D-BiGRU,
verifier)
│   │   └── data/                (dataset loaders)
│   ├── app/                     ← FastAPI layer (routers, schemas, security)
│   ├── pipelines/               ← train / evaluate / calibrate / run_full_pipeline
│   ├── data/research_signatures/ ← recorded research dataset (3 users × 3 sessions)
│   ├── models/                  ← trained .keras models + calibration JSON
│   ├── reports/                 ← evaluation reports (.md/.json)
│   └── tests/                   ← pytest suite
└── scripts/                     ← setup.ps1, start-dev.ps1
```

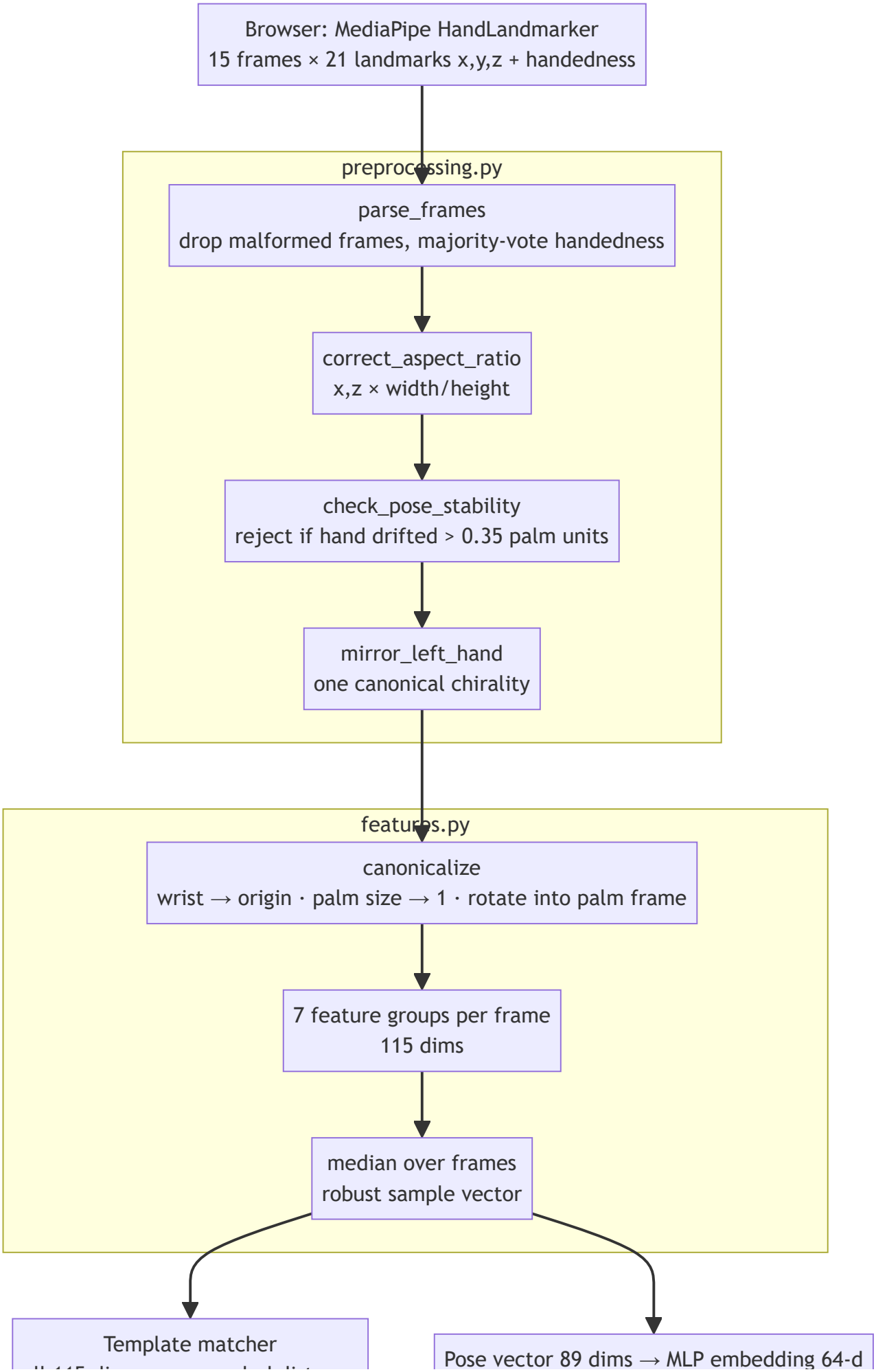
Step 1 — Static Hand Gesture Model

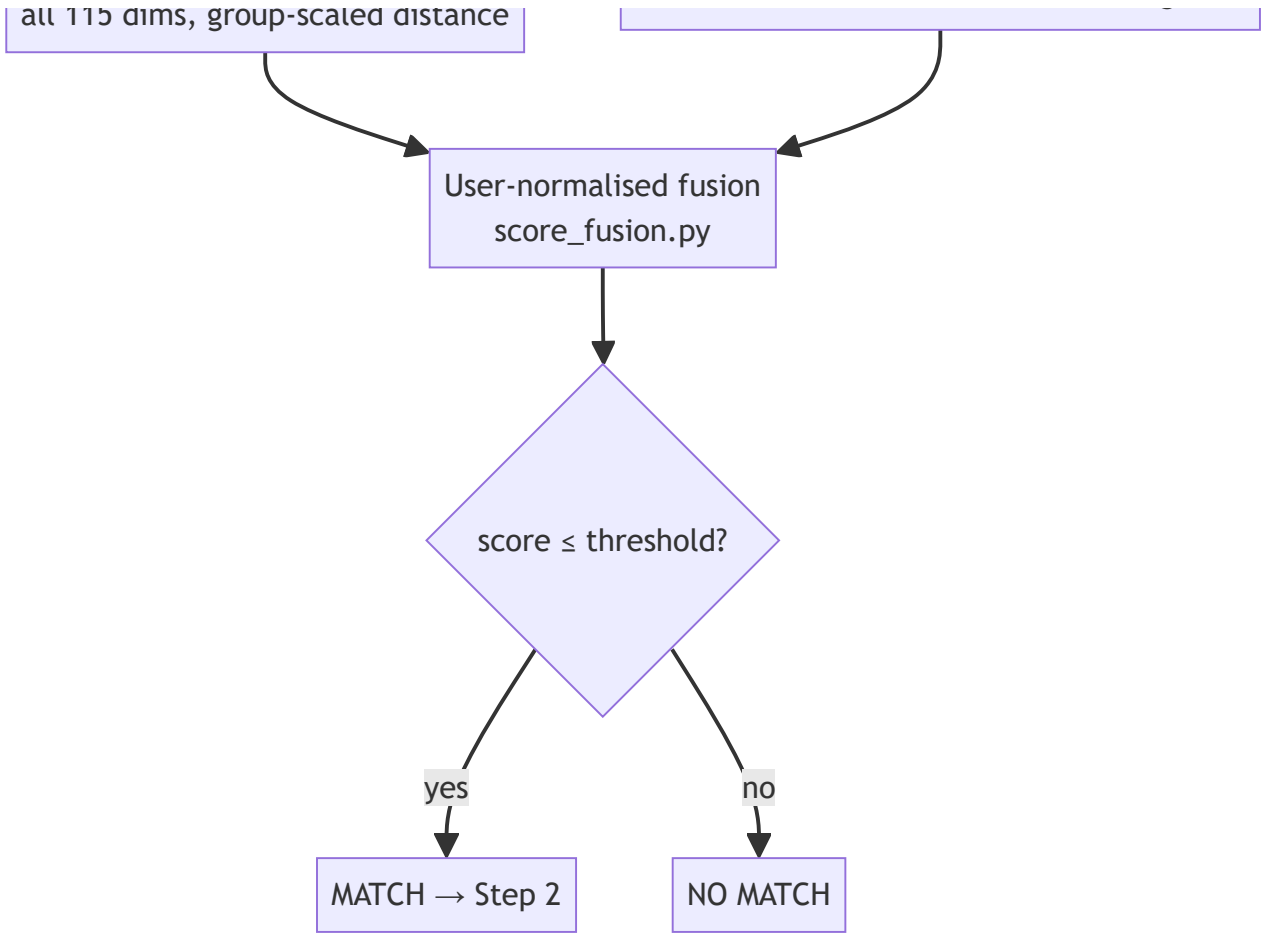
Goal: decide whether the hand pose shown now is the user's registered secret gesture, shown with the same hand.

Code: `ml-service/airauth/static_gesture/`

---

1. Processing pipeline


Browser: MediaPipe HandLandmarker
15 frames × 21 landmarks x,y,z + handedness
preprocessing.py
parse_frames
drop malformed frames, majority-vote handedness
correct_aspect_ratio
x,z × width/height
check_pose_stability
reject if hand drifted > 0.35 palm units
mirror_left_hand
one canonical chirality
features.py
canonicalize
wrist → origin · palm size → 1 · rotate into palm frame
7 feature groups per frame
115 dims
median over frames
robust sample vector
Template matcher
Pose vector 89 dims → MLP embedding 64-d



2. Key-point features (115 per frame)

The palm frame: **y** = wrist → middle knuckle, **x** = index knuckle → pinky knuckle (orthogonalised), **z** = x × y (palm normal). All lengths are in *palm units* (mean wrist-to-knuckle distance), so features do not depend on distance to camera, hand position or in-plane rotation.

Group	Dims	Description	Invariant to	Weight	Scale
canonical_xy	40	x,y of the 20 non-wrist landmarks in the palm frame	position, size, rotation	1.0	0.12
canonical_z	20	depth in the palm frame (MediaPipe z is noisy)	position, size, rotation	0.3	0.30
joint_angles	15	flexion at every finger joint (0 = straight)	everything	1.0	0.22 rad
tip_distances	10	all fingertip-to-fingertip distances	position, size, rotation	1.0	0.15
abduction	4	spread between adjacent fingers (palm plane)	everything	0.5	0.15 rad
orientation	6	palm normal + up vector in camera space (thumbs-up ≠ thumbs-down)	position, size	0.6	0.30
hand_geometry	20	bone length / palm size — **who** the hand belongs to	pose	0.4	0.10

Scale = expected natural variation of a genuine repeat; a group's RMS difference divided by its scale is ≈ 1 for a genuine repeat.

3. AI model — pose embedding network

```
pose vector (89) — standardise (training mean/std)
  → Dense 256 (no bias) → BatchNorm → ReLU → Dropout 0.1
  → Dense 256 (no bias) → BatchNorm → ReLU → Dropout 0.1
  → Dense 128 → ReLU
  → Dense 64 → UnitNormalization                    = embedding (‖e‖ = 1)

training only:  embedding → CosFace head (s = 30, m = 0.35) → softmax over pose classes
```

- **Why metric learning, not a classifier?** Every user invents their own gesture, so there is no fixed class list. CosFace forces a cosine margin between *any* two different poses; at inference we only compare cosine distances, so it works for gestures never seen in training.
- **Training data:** `synthetic_hand.py`, a forward-kinematics hand model. 2,048 pose classes = 4 finger states (extended / half / curled / hooked) ^4 fingers × 2 spreads × 4 thumb states, each with a class-specific offset. Each sample varies the **person** (bone lengths, palm width), **viewpoint** (±35° roll/yaw, ±30° pitch), per-capture joint jitter and MediaPipe-like noise. 50 % of samples are mirrored (chirality augmentation). 80 % of pose classes train the model, 20 % are held out.
- Real captures exported from the app can be added (`--collected`), one class per user.

4. Verification & decision

For enrollment vectors $E_1 \ldots E_N$ (N = 5) and probe P:

Component	Raw distance	User reference (leave-one-out)
template	group-scaled distance(P, mean(E))	mean over i of distance(E_i, mean(E without i))
embedding	1 − cos(emb(P), centroid(emb(E)))	same, leave-one-out

```
effective_ref_c = max(ref_c, floor_c)^0.5 × prior_c^0.5     (shrinkage to population prior)
normalised_c    = raw_c / effective_ref_c
fused           = 0.6 · template + 0.4 · embedding
ACCEPT  ⇔  fused ≤ threshold          (models/static_gesture_calibration.json)
```

Hard rejections before scoring: **wrong hand** (handedness differs from enrollment) and **replay** (probe identical to a stored sample). Enrollment rejects a sample whose distance to the others exceeds 2.5 typical variations ("show exactly the same gesture every time").

5. Results (`ml-service/reports/static_gesture_evaluation.md`)

Simulated users choosing gestures from the 410 **held-out** pose classes, 5 enrollment captures, 4 genuine probes each.

Comparison	EER	ROC-AUC
vs random impostors (other person, own gesture)	**0.1 %**	1.000
template component alone	1.2 %	0.999
embedding component alone	0.2 %	1.000
vs shoulder-surfer (other person copies the gesture)	37.7 %	0.645

Calibrated threshold **3.42** (= min(FAR ≤ 1 % bound, 1.25 × genuine 99th percentile)); at this threshold FAR = 0.1 %, FRR = 0.3 % (synthetic).

Interpretation. Step 1 reliably proves *"this person knows the secret gesture"*. It is deliberately **not** relied on to stop someone who has seen the gesture — hand geometry alone only weakly separates people (AUC 0.65). That is the job of Step 2. Synthetic results are an upper bound; re-run `evaluate_static_gesture_model` after collecting real users, and the calibration will automatically switch to real data.

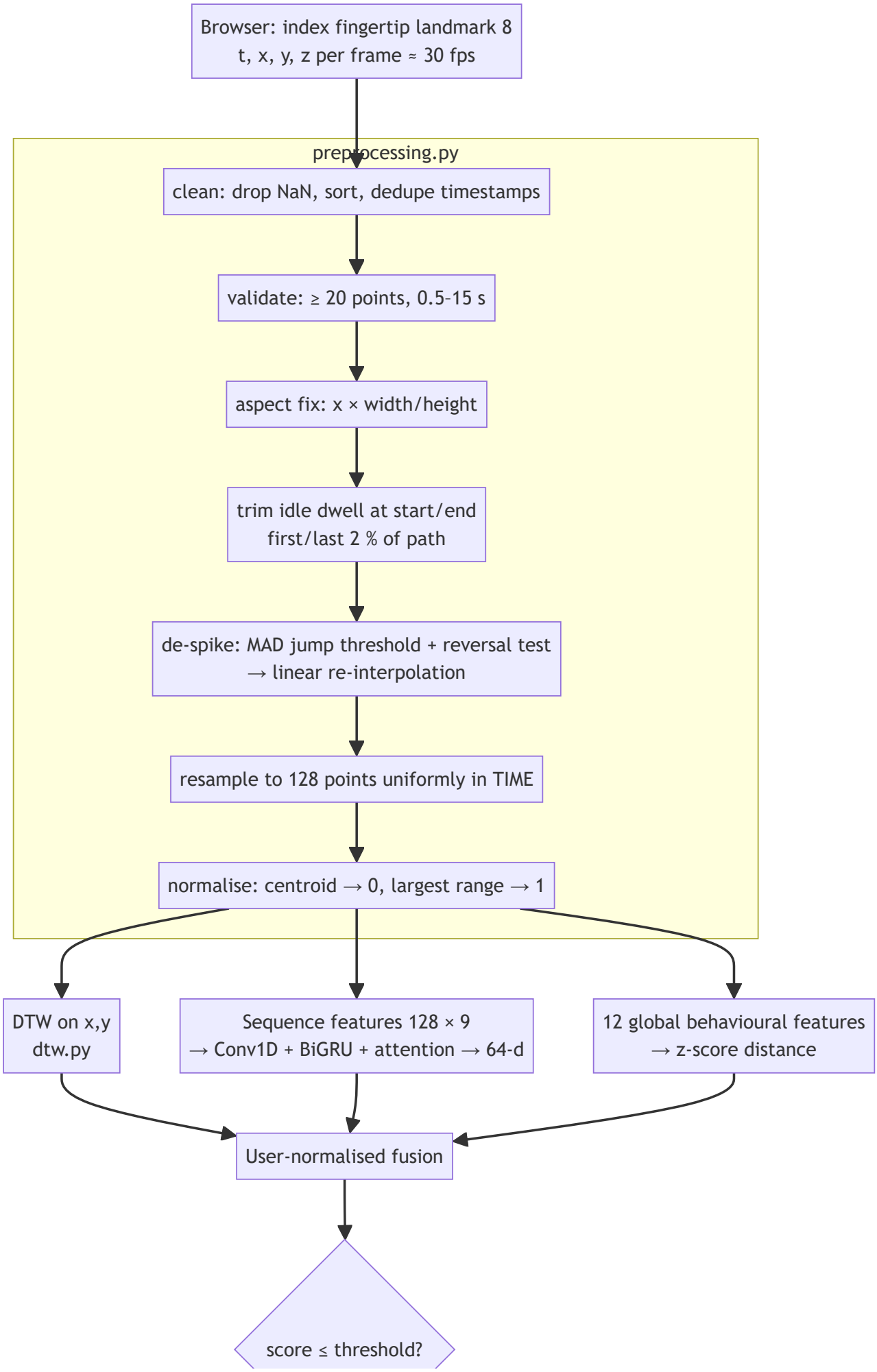
Step 2 — Dynamic Air-Signature Model

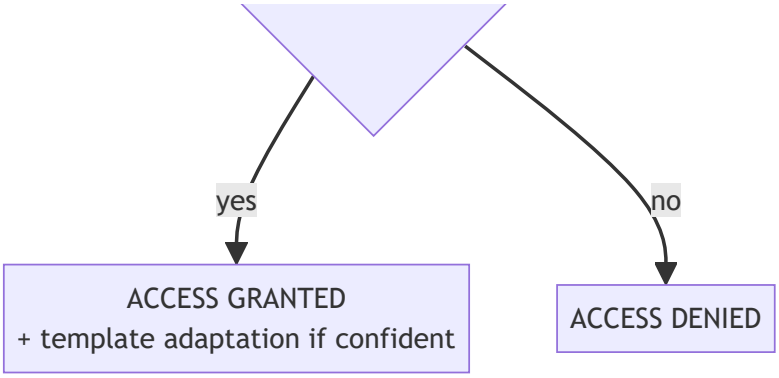
Goal: decide whether the signature just drawn in the air was drawn by the account owner — using *how* it is drawn (speed, rhythm, timing), not only its shape.

Code: `ml-service/airauth/dynamic_signature/`

---

1. Processing pipeline


Browser: index fingertip landmark 8
t, x, y, z per frame ≈ 30 fps
preprocessing.py
clean: drop NaN, sort, dedupe timestamps
validate: ≥ 20 points, 0.5–15 s
aspect fix: x × width/height
trim idle dwell at start/end
first/last 2 % of path
de-spike: MAD jump threshold + reversal test
→ linear re-interpolation
resample to 128 points uniformly in TIME
normalise: centroid → 0, largest range → 1
DTW on x,y
dtw.py
Sequence features 128 × 9
→ Conv1D + BiGRU + attention → 64-d
12 global behavioural features
→ z-score distance
User-normalised fusion
score ≤ threshold?



Resampling **in time** (not arc length) keeps the user's rhythm: fast strokes become sparse, slow ones dense, so DTW and the network both see timing. `z` is dropped — the research ablation showed XY-only DTW is best (EER 14 % vs 31 % with z).

2. Features

Sequence features (per timestep, 9 channels) — computed on a lightly smoothed copy so derivatives are stable:

Channel	Meaning
x, y	normalised position (shape)
vx, vy	velocity w.r.t. normalised time (direction + rhythm)
speed	‖v‖
sin θ, cos θ	pen direction (continuous, no wrap-around)
curvature	dθ/ds, clipped to ±50 (loops, corners)
accel	‖dv/dt‖ (motor dynamics)

Global behavioural features (12): log duration, log path length, log aspect ratio, speed coefficient of variation, peak/mean speed, pause ratio, x- and y-direction reversals, start (x, y), end (x, y).

3. AI model — signature embedding network

```
input 128 × 9 (standardised per channel)
  → Conv1D 64, k5 → BN → ReLU          local stroke shape / velocity patterns
  → Conv1D 64, k5 → BN → ReLU
  → Bidirectional GRU 64 (sequences)   long-range order and rhythm
  → attention pooling                  Dense(1) → softmax over time → weighted sum
  → Dense 128 → ReLU → Dropout 0.2
  → Dense 64 → UnitNormalization       = embedding

training only: CosFace head (s = 30, m = 0.35) over writer identities
```

- **Synthetic pretraining** (`synthetic_signature.py`): 400 writers + 400 skilled forgers × 12 renderings. A writer = control polygon (random walk, or a closed figure 30 % of the time) through a Catmull-Rom spline, a tempo, and a personal rhythm from the **2/3 power law** of human motor control (v ∝ curvature^-β) with writer-specific modulation. Each rendering adds shape jitter, affine change, tempo/rhythm noise, camera frame-rate jitter, tracking noise, spikes and idle dwell.

- **Skilled forgers** copy a writer's shape with their own rhythm and are separate classes — the network must learn dynamics, not just shape.
- The real research dataset is **kept out of training** so it can be used honestly for evaluation and calibration. `--finetune-real` adds it (and web-app exports) once more users exist.
- Inference is compiled to a TensorFlow graph (`tf.function`) — eager BiGRU calls take ~1.9 s, the graph ~45 ms.

4. Verification & decision

Component	Raw distance	Weight
dtw	**min** DTW distance to the enrollment signatures (Sakoe-Chiba band 15 %)	0.45
embedding	1 − cos(probe, centroid of enrollment embeddings)	0.35
global	RMS z-score of the 12 behavioural features (std floored)	0.20

Each component is divided by `sqrt(user_reference × population_prior)`, where the user reference is the leave-one-out distance among their own enrollment samples. The fused score is compared to the calibrated threshold (`models/dynamic_signature_calibration.json`).

Design decisions backed by the research data (3 users × 3 sessions):

Choice	Alternatives tested	Result (EER, 5 → later sessions)
min DTW to templates	mean of 3 nearest / mean of all	18 % vs 25–35 % vs 36–49 %
partial user normalisation (blend 0.5)	none / full	best or tied in every protocol
8 enrollment signatures	5 / 10 / 20 (two sessions)	18 % → 8 % → 3–6 %

Template adaptation: a login accepted with score ≤ 0.75 × threshold is appended to the template (original enrollment always kept, max 20). This reproduces the multi-session gain above over time.

Replay protection: a probe whose DTW distance to a stored sample is below 3 % of the user's natural variation is rejected — humans never reproduce a trajectory exactly.

Enrollment check: a sample whose mean DTW to the others exceeds 4× the median is rejected. On real data the worst genuine sample reached 3.54×, while an inserted *different* person had a median of only 2.3×. So this is a safety net for botched captures, not an impostor detector.

5. Results (`ml-service/reports/dynamic_signature_evaluation.md`)

Real research dataset — 3 users, 90 signatures, impostors are other users' own signatures (random forgeries).

Protocol	DTW	Embedding	Global	**Fused**
web registration (8 from session 1 → sessions 2–3)	13.3 %	26.7 %	21.7 %	**8.3 %** (AUC 0.943)
multi-session (2 sessions → 3rd, 3 folds)	14.4 %	18.9 %	13.3 %	14.4 % (AUC 0.899)

Operating point (threshold 1.253, target FAR ≤ 5 %): FAR 5.0 %, FRR 16.7 % per attempt (web registration). With 3 attempts per step a genuine user is locked out of a session with probability ≈ 0.5 %.

Live HTTP check (8-signature enrollment, every later-session signature): genuine accepted 50/60, impostors accepted 5/120. Verification latency ≈ 140 ms (8 templates) to ≈ 470 ms (20 templates).

Synthetic held-out writers: EER 1.5 %; synthetic skilled forgeries: EER 19.6 %.

Honest limitations. Three users is far too few for reliable numbers; the research "signatures" are simple shapes (circle, square, triangle), which are much easier to confuse than real names. The embedding shows a synthetic → real gap (pairwise EER 32 % on real data alone) and adds only a little today: the best fusion without it reaches 8.7 % EER, with it 8.3 %. Both will improve once real users register: run `npm run export-samples` and then `run_full_pipeline --use-collected`.

ML Pipelines — Training, Evaluation, Calibration, Retraining

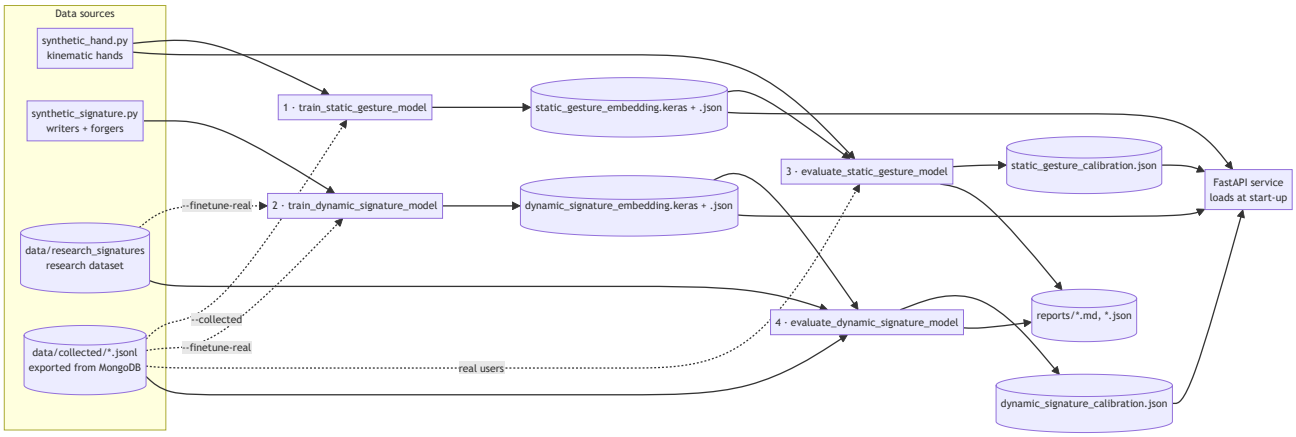
All pipelines live in `ml-service/pipelines/` and are run from `ml-service/` with the virtual environment's Python:

```
cd ml-service
.\.venv\Scripts\python.exe -m pipelines.<name> [options]
```

They reuse the **exact production code** (preprocessing, features, `component_scores`, fusion) so offline metrics match live behaviour.

---

1. Pipeline map



`run_full_pipeline` runs 1 → 2 → 3 → 4 in order (evaluation must follow training: thresholds depend on the exact weights).

2. Pipelines

#	Module	Inputs	Outputs	Default runtime (CPU)
1	`train_static_gesture_model`	1,638 synthetic pose classes × 40	`models/static_gesture_embedding.{keras,json}`	≈ 4 min
2	`train_dynamic_signature_model`	400 writers + 400 forgers × 12	`models/dynamic_signature_embedding.{keras,json}`	≈ 7 min

#	Module	Inputs	Outputs	Default runtime (CPU)
3	`evaluate_static_gesture_model`	150 simulated users (held-out poses); real exports if ≥ 2 users	`models/static_gesture_calibration.json`, report	≈ 1 min
4	`evaluate_dynamic_signature_model`	research dataset + exports	`models/dynamic_signature_calibration.json`, report	≈ 2.5 min
—	`run_full_pipeline`	—	all of the above	≈ 15 min

Useful options:

```
train_static_gesture_model      --instances 40 --epochs 25 --collected
train_dynamic_signature_model   --writers 400 --renders 12 --epochs 18 --finetune-real
evaluate_static_gesture_model   --users 150 --target-far 0.01 --target-frr 0.01 --genuine-margin
1.25
evaluate_dynamic_signature_model --enroll 8 --target-far 0.05 --apply-best-weights
run_full_pipeline               --quick | --skip-training | --use-collected
```

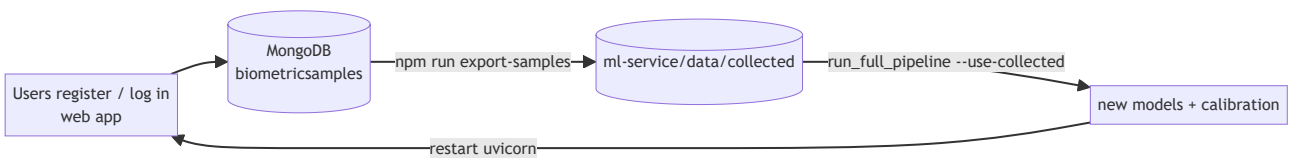
3. What each artefact contains

File	Content	Used by
`*_embedding.keras`	embedding network only (no CosFace head)	`model_registry.py`
`*_embedding.json`	model version, input mean/std, training size, validation EERs	`model_registry.py`, reports
`*_calibration.json`	threshold + policy, fusion weights, reference priors, data source	`config.load_*_calibration()` on every verification
`reports/*_evaluation.md`	human-readable metric tables	documentation / capstone report

Threshold policies:

- **Static:** `min(FAR ≤ 1 % bound, 1.25 × 99th percentile of genuine scores)` — anchored on genuine variation, because synthetic impostors are so well separated that a FAR-only rule would be far too loose.
- **Dynamic:** loosest threshold with FAR ≤ 5 % on the web-registration protocol; falls back to the EER point if that would reject > 50 % of genuine attempts.

4. Continuous improvement loop (real users)



Only enrollment samples and **accepted** login samples are exported (their identity label is trustworthy). Session ids are per day, so the dynamic evaluation automatically gets a realistic multi-session protocol. Templates store processed features rather than embeddings, so **retraining never invalidates existing users**; only a change of `*_FEATURE_VERSION` in `config.py` requires re-registration.

5. Tests

```
.\.venv\Scripts\python.exe -m pytest tests -q
```

23 tests: feature invariance, genuine/impostor decisions, wrong hand, replay, enrollment consistency, capture validation, DTW properties, template adaptation, API auth / round trips / error format.

Project Flow — What To Do, In Order

A. One-time setup (≈ 15 min)

#	Task	Command / action
1	Install **Node.js 18+** (not installed on this machine yet)	`winget install OpenJS.NodeJS.LTS` — then open a new terminal
2	Install / provide **MongoDB** — pick one	`winget install MongoDB.Server` (runs as a Windows service) · or `docker run -d -p 27017:27017 --name airauth-mongo mongo:7` · or a free MongoDB Atlas cluster
3	Install all dependencies	`powershell -ExecutionPolicy Bypass -File scripts\setup.ps1`
4	Configure the backend	edit `server\.env` (created from `.env.example`): `MONGODB_URI`, a long random `JWT_SECRET`
5	(Optional) retrain + recalibrate models	`cd ml-service; .\.venv\Scripts\python.exe -m pipelines.run_full_pipeline` — trained models are already included

The ML service already has its virtual environment (`ml-service\.venv`) and trained models in `ml-service\models\`.

B. Run the system

```
powershell -ExecutionPolicy Bypass -File scripts\start-dev.ps1
```

Or manually, in three terminals:

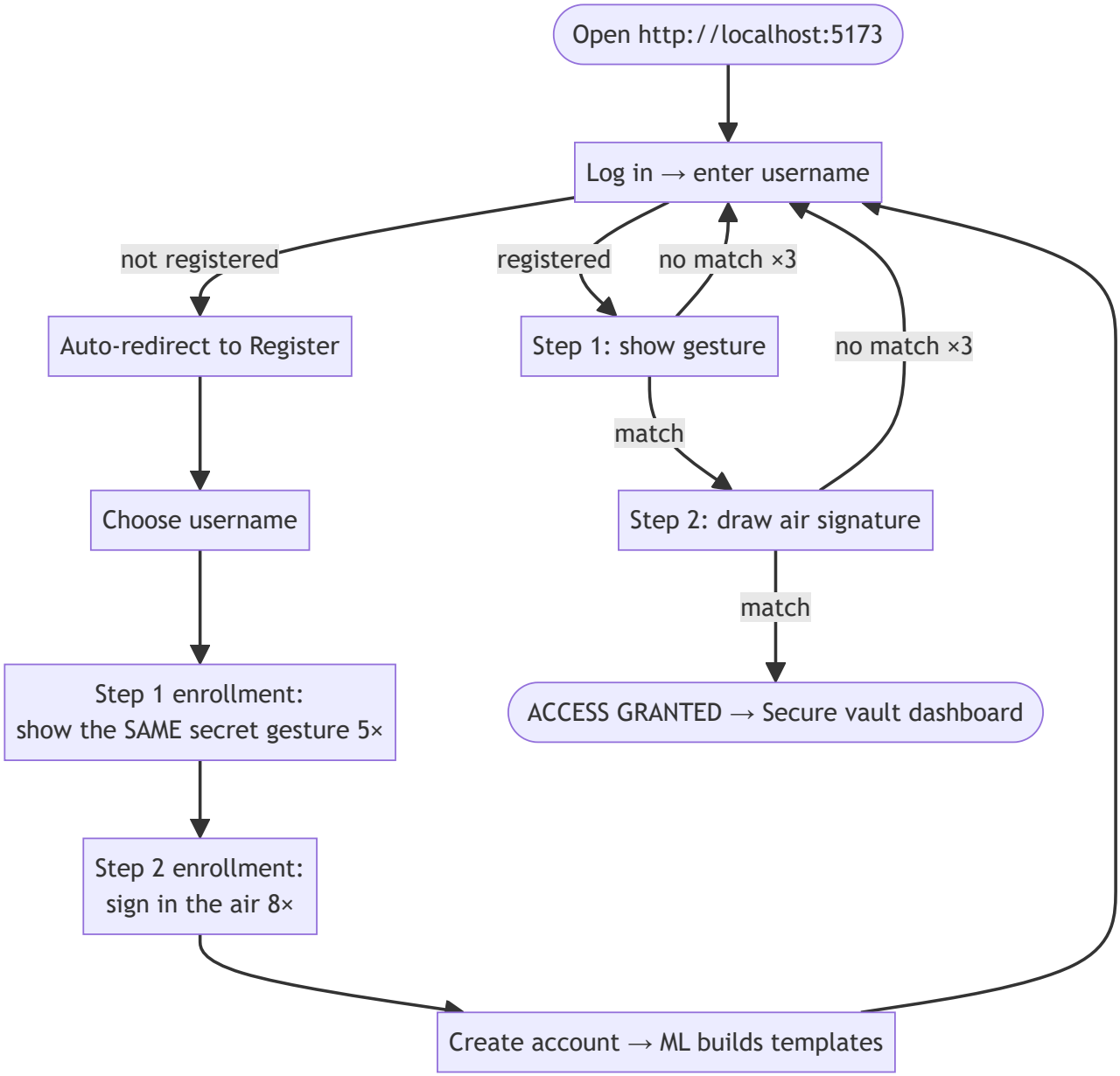
```
# 1. ML service  (http://127.0.0.1:8000/docs)
cd ml-service; .\.venv\Scripts\python.exe -m uvicorn app.main:app --port 8000

# 2. API         (http://localhost:5000/api/system/health)
cd server; npm run dev

# 3. Web app     (http://localhost:5173)
cd client; npm run dev
```

Start order matters only loosely: the API needs MongoDB at start-up; the ML service can start any time (the UI status panel shows if it is down).

C. End-to-end user flow



D. Capstone demonstration script

1. Home page → status panel shows API / MongoDB / ML service and model versions.
2. **Register** "demo_user": 5 gesture captures (e.g. thumb + pinky out), 8 air signatures.
3. **Log in** as demo_user → gesture ✓ (result card with match % and component scores) → signature ✓ → **ACCESS GRANTED**, dashboard.
4. **Impostor A** (another person) logs in as demo_user with a different gesture → Step 1 rejected.
5. **Impostor B** is shown the gesture (shoulder-surfing) → passes Step 1 → draws the signature with their own motion → **Step 2 rejected**. This is the key point of the two-step design.
6. Fail 5 times → account lock (423) is demonstrated.
7. Dashboard → authentication history shows every attempt with scores.
8. Show `ml-service/reports/*.md` for EER / ROC-AUC / FAR / FRR and the architecture docs.

E. Development roadmap (what remains / next steps)

Priority	Item	Why
1	**Collect real users** (10–20 people, 2–3 sessions each) through the app; `npm run export-samples`; `run_full_pipeline --use-collected`	All current thresholds come from 3 research users + synthetic data
2	Collect **skilled forgeries** (people imitating a signature after watching it)	Only random forgeries are measured on real data today
3	Fine-tune the signature embedding on real data (`--finetune-real`) once ≥ 10 users exist	Closes the synthetic → real gap
4	Liveness / challenge step (e.g. "draw a random digit first")	Client-side landmarks could be forged by a malicious client
5	HTTPS deployment (camera access requires a secure origin outside localhost)	Production
6	Model registry / A-B thresholds per deployment	Operations

F. Where everything is

Need	File
System overview, security	`Architecture/01_System_Architecture.md`
Step 1 model	`Architecture/02_Static_Gesture_Model.md`
Step 2 model	`Architecture/03_Dynamic_Signature_Model.md`
Training / calibration	`Architecture/04_ML_Pipelines.md`
Original research plan	`Architecture/legacy_research_notes/`
Measured metrics	`ml-service/reports/*_evaluation.md`

AirAuth — The AI Explained: What We Used, How It Works, and Why

This document explains, in plain language, every technology and AI technique inside AirAuth: **what** each part is, **what it does** in the system, **why we chose it**, and **why it is better than the obvious alternatives**. All numbers come from the project's own evaluation reports (`ml-service/reports/`).

Contents

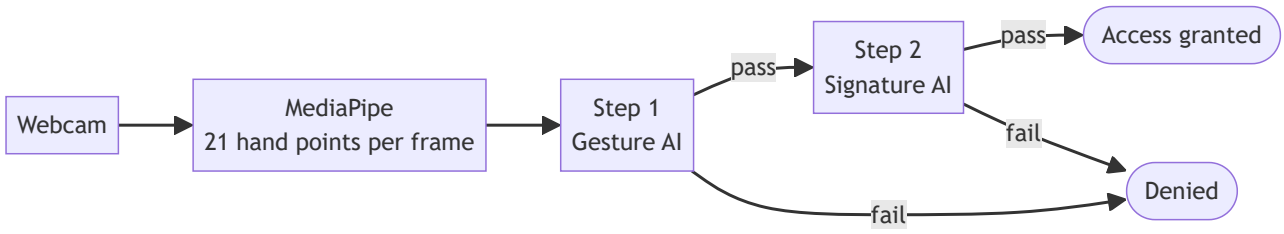
1. The big picture
2. Seeing the hand — MediaPipe
3. Step 1: the static gesture
4. Step 2: the air signature
5. How the models learn — metric learning with CosFace
6. Synthetic training data — why and how
7. Making the decision — score fusion and thresholds
8. Security techniques
9. How we measure accuracy
10. The software stack and why
11. Summary table — every choice at a glance
12. Honest limitations
13. Glossary

---

1. The big picture

AirAuth logs a person in with **two factors**, both captured by an ordinary webcam:

Step	Factor	Type of factor	Question it answers
1	**Static hand gesture** (a secret hand pose)	Something you *know* (+ a little of who you *are*)	"Does this person know the secret pose?"
2	**Dynamic air signature** (signing in the air with a finger)	Something you *are* — a behavioural biometric	"Does this person **move** like the account owner?"



Why two steps? Each step covers the other's weakness:

- A gesture is easy to *copy* if someone watches you (shoulder-surfing), but it is fast and very reliable at rejecting random people.
- A signature's *motion* (speed, rhythm, pauses) is very hard to copy even after watching it, but on its own it has a higher error rate.

Together, an attacker must know the secret **and** move like you.

---

2. Seeing the hand — MediaPipe

What it is

MediaPipe HandLandmarker is a pre-trained Google neural network that finds a hand in an image and returns **21 key-points (landmarks)** — the wrist plus four points on each finger — each with x, y and a relative depth z.

```
        8   12  16  20        ← fingertips
        |   |   |   |
        7   11  15  19
    4   6   10  14  18
    |   5---9---13--17        ← knuckles
    3  /
     2/
      1
       \___ 0 (wrist)
```

What it does in AirAuth

It runs **inside the browser** (WebAssembly, GPU when available) on every video frame. Only the 21 × 3 numbers are sent to the server — never the video.

Why we chose it

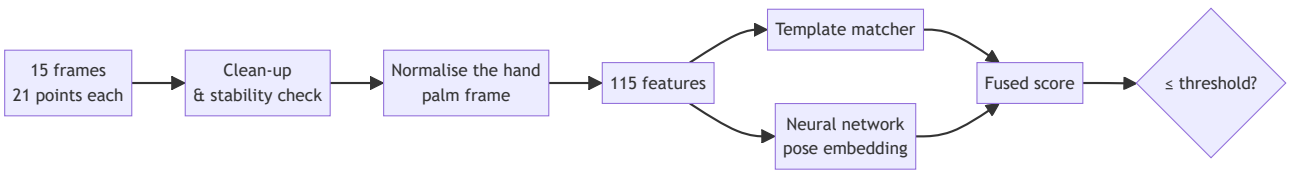
Alternative	Problem	Why MediaPipe is better
Train our own hand detector (e.g. a CNN on images)	Needs tens of thousands of labelled hand images and a GPU	MediaPipe is already trained on huge datasets and is accurate out of the box
Send video to the server and process it there	Slow, expensive, and a **privacy risk** (faces, rooms on camera)	On-device processing: private, no video upload, low bandwidth
Depth cameras / Leap Motion / gloves	Special hardware	Works with any laptop webcam
OpenPose and similar	Heavy, built for full body, slow on CPU	MediaPipe is lightweight and real-time (~30 fps) in a browser

Bonus: working with key-points instead of raw pixels makes the AI models tiny and fast, and immune to lighting, skin colour or background — those were already handled by MediaPipe.

---

3. Step 1: the static gesture

3.1 What goes on



1. **Capture 15 frames** while the user holds the pose (~0.5 s) and take the **median** of each feature over the frames. *Why?* One frame can be noisy; the median ignores the odd bad frame.
2. **Stability check** — reject if the hand moved while "holding still". *Why?* A moving hand means the pose was not held or tracking failed.
3. **Normalise the hand** (the most important step): * move the wrist to the origin → position no longer matters * divide by palm size → distance from camera no longer matters * rotate into the **palm's own coordinate frame** → tilting/rotating the hand no longer matters * mirror left hands → one consistent layout

Why? The same gesture must look identical to the AI whether your hand is near or far, left or right in the picture, or slightly tilted.

3.2 The 115 features — and why each group exists

Group	Count	What it measures	Why it helps
Canonical x, y	40	Where each point is relative to the palm	The overall shape of the pose
Canonical z	20	Depth of each point	Fingers bent towards/away from camera (low weight: depth is noisy)
Joint angles	15	How bent each finger joint is	The single best description of a pose — completely independent of hand size and position
Fingertip distances	10	Distances between all fingertip pairs	Captures "touching" gestures (e.g. OK sign)
Abduction angles	4	Spread between neighbouring fingers	Separates "fingers together" from "fingers spread"
Orientation	6	Which way the palm faces / points	Distinguishes thumbs-up from thumbs-down (same shape, different direction)
Hand geometry	20	Bone lengths ÷ palm size	Your finger proportions — a weak *biometric* (whose hand it is)

Why hand-crafted features instead of feeding raw points to a network? Raw coordinates change completely when you move your hand; the network would have to learn all those invariances from a lot of data. Engineered features encode the invariances directly, so the system works with only **5 enrollment samples** per user.

3.3 Model A — the template matcher

Compares the probe's features with the average of the user's 5 enrollment samples, group by group, each group scaled by how much it naturally varies (e.g. joint angles normally vary ~12°).

Why keep a simple, non-neural matcher? It is transparent, needs no training, works for any gesture, and acts as a safety net if the neural model is unavailable.

3.4 Model B — the pose embedding network (MLP)

```
89 pose features
 → Dense 256 → BatchNorm → ReLU → Dropout 0.1
 → Dense 256 → BatchNorm → ReLU → Dropout 0.1
 → Dense 128 → ReLU
 → Dense 64  → Unit-length normalisation   = "embedding"
```

131,520 parameters · trained on 65,520 synthetic samples of 1,638 pose classes.

What it does: turns a pose into a 64-number "fingerprint" (embedding). Same pose → nearby fingerprints; different pose → far apart. We compare fingerprints with **cosine distance**.

Why an MLP (multi-layer perceptron)?

Alternative	Why not
CNN on images	Needs images and far more data; MediaPipe already did the "vision" part
Graph neural network on the hand skeleton	More complex; with good engineered features an MLP reaches EER 0.07 % on unseen poses — little room left to gain
Classical classifier (SVM, Random Forest)	A classifier needs a **fixed list of classes**; every user invents their own gesture, so there is no fixed list
Just the template matcher	Works, but the network learned *which differences matter* (e.g. ignore small depth noise, care about a curled finger), and halves the error (1.2 % → 0.1 % EER fused)

BatchNorm keeps training stable; **Dropout** prevents memorising the training poses; **unit-length normalisation** makes cosine distance the natural comparison.

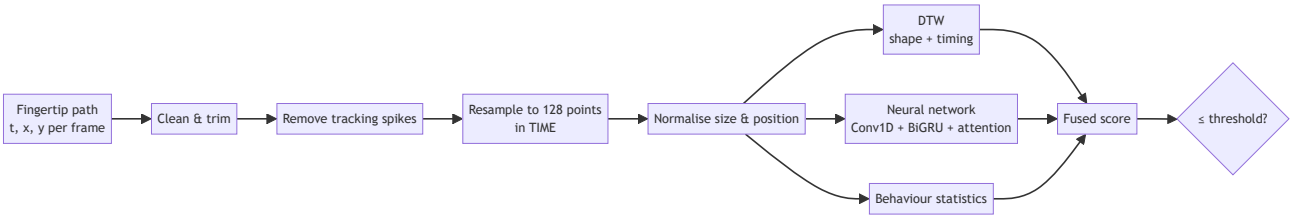
3.5 Results

Test (150 simulated users, gestures never seen in training)	Error (EER)
Template matcher alone	1.2 %
Neural embedding alone	0.2 %
Both fused	**0.1 %**
Someone who *saw and copies* your gesture	37.7 % (expected — that is Step 2's job)

---

4. Step 2: the air signature

4.1 What goes on



4.2 Preprocessing — and why each step matters

Step	What	Why
Clean	Drop broken points, sort by time	Camera data is never perfect
Trim idle	Remove the still periods at start/end	The recorder waits for your finger to rest; that pause is not part of the signature
Spike removal (MAD test)	Detect sudden jumps that immediately reverse, replace by interpolation	MediaPipe occasionally "jumps" for one frame; a genuine fast stroke keeps going, a glitch comes back — so we only fix glitches and keep real speed

Step	What	Why
Resample in time to 128 points	Equal time steps	Every signature becomes the same length *and* the spacing of points still encodes **speed** (fast strokes → points far apart). Resampling by distance would throw the rhythm away
Normalise	Centre and scale to size 1	Signing bigger/smaller or further left/right must not matter
Drop z (depth)	Use only x, y	Measured: depth is noisy — XY-only gave 14 % error vs 31 % with depth

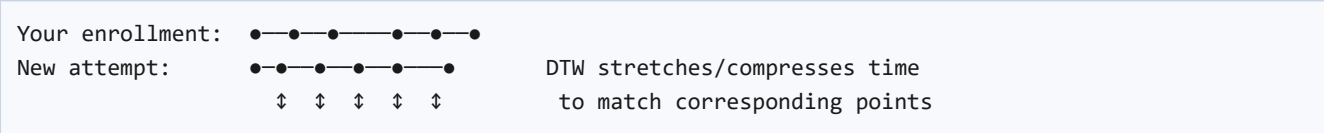
4.3 Features

9 values per time step (for the neural network): position (x, y), velocity (vx, vy), speed, direction as sin/cos, curvature, acceleration. *Why sin/cos for direction?* An angle jumps from 359° to 0°; sin/cos is smooth, so the network isn't confused.

12 behaviour statistics (one set per signature): duration, path length, width/height ratio, speed variability, peak speed, pause ratio, number of left/right and up/down reversals, start and end points. *Why?* These describe your **habits** — how long you take, where you pause — which an imitator rarely reproduces.

4.4 Model A — DTW (Dynamic Time Warping)

What: an algorithm that aligns two sequences that may be locally faster or slower, then measures how different they are.



Why DTW?

Alternative	Problem	DTW advantage
Point-by-point (Euclidean) comparison	If you sign 10 % slower in one part, every later point is "wrong"	DTW tolerates natural tempo changes
Comparing summary numbers only	Loses the shape	DTW compares the full trajectory
Deep model only	Needs lots of real data we don't have yet	DTW needs **no training** and works from day one

Design choices measured on the real dataset: * compare against the **closest** enrollment signature (min) — error 18 % vs 25–35 % (mean of 3 nearest) vs 36–49 % (mean of all), with 5 samples; * a **Sakoe-Chiba band** (15 %) forbids absurd alignments and speeds it up; * **batched DTW** — we compute many comparisons at once with NumPy, cutting 190 comparisons from ~1 s to 0.18 s.

4.5 Model B — the signature embedding network

```
128 time steps × 9 features
 → Conv1D 64 (kernel 5) → BatchNorm → ReLU
 → Conv1D 64 (kernel 5) → BatchNorm → ReLU
 → Bidirectional GRU (64)
 → Attention pooling
 → Dense 128 → ReLU → Dropout 0.2
 → Dense 64  → Unit-length normalisation    = "signature fingerprint"
```

98,689 parameters · trained on 9,600 synthetic signatures of 800 identities.

Each layer has a job:

Layer	Job	Why this layer
Conv1D (1-D convolution)	Detects short local patterns: a hook, a sharp corner, a quick flick	Convolutions are the standard, efficient way to find local patterns anywhere in a sequence
Bidirectional GRU	Reads the whole signature forwards *and* backwards, remembering order and rhythm	A recurrent network models *time*; "bidirectional" lets each moment see what came before and after. GRU is lighter and trains faster than LSTM with similar accuracy
Attention pooling	Learns which moments of the signature are most identifying and weights them more	Better than just taking the last step (which forgets the beginning) or a plain average (which treats boring and distinctive parts equally)
Dense + normalise	Compresses to a 64-number fingerprint	Compare by cosine distance, like Step 1

Why this hybrid (CNN + RNN + attention) instead of…

Alternative	Why not
LSTM only (the original research baseline)	Used the last time step only and was trained as a 3-class classifier — it cannot verify *new* users
Transformer	Needs much more data; for 128-step sequences a GRU is accurate and far cheaper
Pure CNN	Weaker at long-range timing/rhythm
Siamese network on pairs	Same idea (compare two signatures), but CosFace training (section 5) is more stable and data-efficient

4.6 Model C — behaviour statistics

Compares the 12 statistics with the user's enrollment average using z-scores ("how many standard deviations away"), with minimum spreads so a user who happened to be very consistent at enrollment is not rejected for a tiny difference.

4.7 Why three models instead of one?

They make **different kinds of mistakes**: DTW is strong on shape, the statistics on habits, the network on learned patterns. Combining them (see section 7) reduced the error on real data:

Method (8 enrollment signatures, tested on later sessions)	Error (EER)
DTW alone	13.3 %
Behaviour statistics alone	21.7 %
Neural network alone	26.7 %
All three fused	**8.3 %**

---

5. How the models learn — metric learning with CosFace

The problem

A normal classifier learns a fixed set of classes ("cat", "dog"). But in authentication **every new user is a new class** the model has never seen. We cannot retrain the model whenever someone registers.

The solution: learn a *similarity space* (metric learning)

We train the network so that samples of the **same** identity land close together and **different** identities land far apart. At login we don't ask "which class?" — we ask "**how close is this to the user's enrollment samples?**". That works for anyone, including users who registered after training.

Why CosFace specifically

During training only, a **CosFace** head is attached:

```
logit = 30 × ( cos(angle to own class) − 0.35 )      for the true class
logit = 30 ×   cos(angle to other class)             for other classes
```

The **margin 0.35** forces each class to be clearly separated, not just barely. After training the head is thrown away; only the embedding network is kept.

Alternative	Drawback	CosFace advantage
Plain softmax classifier	Classes separated, but without a safety margin — new users blur together	Margin → well-separated, compact clusters
Triplet loss / Siamese pairs	Needs careful selection of "hard" pairs; slow, unstable training	Uses all classes at once in every batch; stable and fast
ArcFace	Very similar; equally valid	CosFace is simpler to implement and tune

CosFace is the same family of methods used in modern **face recognition**, which faces exactly the same "verify people never seen in training" problem.

---

6. Synthetic training data — why and how

Why

Deep networks need many identities. We had **3 real users** — far too few. Collecting hundreds of people was not possible for the prototype. So we **generate realistic artificial data**, and keep the real recordings separate for honest testing.

Synthetic hands (Step 1)

A **forward-kinematics hand model**: a skeleton with realistic bone lengths whose joints are bent by chosen angles.

- **2,048 pose classes** = 4 states per finger (extended, half, curled, hooked) × 2 spreads × 4 thumb positions
- every sample varies the **person** (bone lengths, palm width), the **viewing angle** (±35°), joint jitter and camera noise
- 50 % are mirrored so left and right hands are covered
- **20 % of pose classes are held out** — the model is tested only on gestures it never saw

Synthetic signatures (Step 2)

- a "writer" = random pen path (or a closed shape), a tempo, and a personal **rhythm** based on the **2/3 power law** from motor-control research (people naturally slow down in tight curves) plus individual speed variations
- each repetition adds natural variation, camera frame-rate jitter, tracking noise and glitches
- **skilled forgers**: copy a writer's *shape* but draw it with their own rhythm. They count as different people, which **forces the network to learn motion, not just shape**

Why this is better than training on our 3 real users

A network trained on 3 people learns to tell *those 3* apart and nothing else. Synthetic data gives it **800–1,638 identities**, so it learns the general skill of comparing gestures/signatures. The real data stays untouched for an unbiased evaluation. When real users register, their data can be added (`run_full_pipeline --use-collected`).

---

7. Making the decision — score fusion and thresholds

7.1 Personal normalisation

Some people sign very consistently, others vary a lot. A raw distance of 0.05 might be "normal" for one person and "suspicious" for another. So every distance is divided by **that user's own natural variation** (measured between their enrollment samples, leaving one out at a time), blended 50/50 with a population average:

```
normalised = raw distance / √( user's variation × population average )
```

Why blend? With only 5–8 enrollment samples, the user's own estimate is noisy. Blending ("shrinkage") was the most robust option on real data.

7.2 Fusion

```
Step 1:  score = 0.6 × template  + 0.4 × embedding
Step 2:  score = 0.45 × DTW + 0.35 × embedding + 0.20 × behaviour
```

If the neural model is missing, its weight is redistributed automatically — the system degrades gracefully instead of failing.

7.3 Thresholds (calibrated, not guessed)

The accept/reject threshold is **computed from data** by the evaluation pipelines, not picked by hand:

Step	Rule	Value
1	strictest of: false-accept ≤ 1 %, or 1.25 × the score that 99 % of genuine attempts reach	3.42
2	false-accept rate ≤ 5 % on real signatures	1.25

The UI shows a **match %**: 50 % = exactly at the threshold.

7.4 Template adaptation

When you log in with a **confident** signature match, that signature is added to your profile (original enrollment always kept, up to 20). *Why?* People's signing drifts over days. On real data, more and multi-session samples cut the error from ~18 % (5 samples) to 3–6 % (20 samples from two sessions).

---

8. Security techniques

Technique	What it stops
Two ordered steps, enforced on the server	Skipping the gesture step
Wrong-hand check	Using the other hand
Replay detection	Re-sending a recorded sample — humans never repeat a movement *exactly*, so a near-perfect copy is rejected
3 attempts per step, lock after 5 failures, rate limits	Brute-force guessing
Video stays in the browser; templates stay on the server	Leaking biometric data
Shared service key	Anyone calling the AI service directly
Short-lived JWT tokens, hashed session tokens	Stolen-token misuse

---

9. How we measure accuracy

Metric	Meaning	Good value
FAR — False Acceptance Rate	% of impostors wrongly let in	Low (security)
FRR — False Rejection Rate	% of genuine users wrongly rejected	Low (convenience)
EER — Equal Error Rate	The point where FAR = FRR; one number to compare systems	Lower is better
ROC-AUC	Probability a genuine attempt scores better than an impostor (1.0 = perfect)	Closer to 1

Why not "accuracy"? In authentication impostors and genuine users are not balanced, and letting in an impostor is far worse than asking someone to retry. FAR/FRR/EER are the industry standard for biometrics.

Fair testing: enroll on early sessions, test on **later sessions** (people sign differently on different days), and test the AI on gestures and writers it **never saw** during training.

---

10. The software stack and why

Layer	Technology	Why it was chosen
Hand tracking	MediaPipe Tasks Vision (WASM)	Accurate, real-time, runs in the browser, private
Front-end	React 18 + Vite	Component-based UI for the multi-step wizards; Vite = instant dev server
Back-end API	Node.js + Express	The "E/N" of MERN; simple, fast REST API with rich middleware (security headers, rate limiting)
Database	MongoDB + Mongoose	The "M" of MERN; stores variable-shaped JSON (templates, raw samples) naturally; TTL index auto-deletes expired login sessions
Auth tokens	JSON Web Tokens	Stateless, standard

Layer	Technology	Why it was chosen
AI service	Python + FastAPI	Python is the language of ML; FastAPI is fast, typed and auto-documents the API (`/docs`)
Numerical core	NumPy	Preprocessing, features, DTW and metrics — no heavy dependencies needed
Deep learning	TensorFlow / Keras 3	Mature, runs on CPU, saves models in one file; inference compiled to a graph (`tf.function`) → 45 ms instead of 1.9 s
Testing	pytest + end-to-end API script	23 automated ML tests; 19 full-stack checks

Why a separate Python service instead of doing ML in Node.js? The ML ecosystem (TensorFlow, NumPy) is in Python, and the original research code was Python. Keeping ML in its own service also means it can be retrained, scaled or replaced without touching the website.

---

11. Summary table — every choice at a glance

Component	What we used	Why it's the better choice
Hand detection	MediaPipe (21 landmarks)	Pre-trained, real-time, private, any webcam
Gesture representation	115 engineered, normalised features	Built-in invariance → works with only 5 samples
Gesture AI	MLP embedding + template matcher	Handles gestures never seen in training; matcher as safety net
Signature preprocessing	Spike removal + time resampling	Fixes glitches without destroying speed/rhythm
Signature shape/timing	DTW (min over templates, banded, batched)	Works with zero training, tolerant to tempo changes, fast
Signature AI	Conv1D + BiGRU + attention	Local stroke patterns + rhythm + focus on distinctive parts, small enough for CPU
Behaviour	12 statistics with floored z-scores	Captures habits imitators miss
Training method	CosFace metric learning	Verifies brand-new users without retraining
Training data	Synthetic hands, writers and forgers	Thousands of identities; real data kept for honest testing
Decision	Personal normalisation + weighted fusion	Fair across consistent and inconsistent users; models cover each other's mistakes
Threshold	Calibrated from data	Defensible, reproducible security level
Over time	Template adaptation	Follows natural drift; error drops as samples grow

---

12. Honest limitations

- **Small real dataset:** only 3 real users, who drew simple shapes (circle, square, triangle). Real names are more distinctive, but the numbers need confirming with 10–20+ real users.
- **Synthetic → real gap:** the signature network is excellent on synthetic writers (1.5 % EER) but weaker on real data alone (32 % pairwise). It still helps the fused result slightly (8.7 % → 8.3 %), and will improve when fine-tuned on real users.

- **Static gesture can be copied:** by design, Step 1 is a knowledge factor; Step 2 provides the biometric protection.
- **Skilled forgeries on real data are not yet measured** — only on synthetic forgers (19.6 % EER).
- **Client-side landmarks:** a malicious program could send fake landmarks; production would add liveness challenges.

---

13. Glossary

Term	Meaning
Landmark / key-point	One of the 21 tracked points on the hand
Embedding	A short list of numbers (here 64) that summarises a sample; similar samples → similar embeddings
Cosine distance	How different the *directions* of two embeddings are (0 = identical)
Metric learning	Training a model to produce embeddings where distance = dissimilarity
CosFace	A metric-learning loss that adds a safety margin between classes
DTW	Dynamic Time Warping — compares sequences while allowing local speed differences
GRU / BiGRU	A recurrent neural network layer that reads sequences step by step; "Bi" = both directions
Conv1D	A convolution that slides along a sequence to detect local patterns
Attention pooling	A learned weighted average that focuses on the most useful time steps
MLP	Multi-layer perceptron — a stack of fully connected layers
Enrollment	Registering your gesture/signature samples
Template	The stored processed version of your enrollment samples
FAR / FRR / EER	False accept rate / false reject rate / equal error rate
Calibration	Choosing the decision threshold from measured data